

Lecture 2

Solving Problems by Searching

TDT4136: Introduction to Artificial Intelligence

Massimiliano Ruocco

Department of Computer Science
Faculty of Information Technology and Electrical Engineering
Norwegian University of Science and Technology

August 27, 2026

Outline

1 Problem solving and search

2 The search algorithm

3 Uninformed search strategies

- Breadth-first search

- Depth First Search

- Depth-limited and Iterative deepening search

- Uniform-cost search (Dijkstra)

4 Informed search strategies

- (Greedy) Best First Search

- A^* Search

Why searching?

Problem solving and search

- ▶ When the **correct action** to take is not immediately obvious, an agent may need to plan ahead

Why searching?

Problem solving and search

- ▶ When the **correct action** to take is not immediately obvious, an agent may need to **plan ahead**
- ▶ That means considering a **sequence of actions** that form a path to a **goal state**

Why searching?

Problem solving and search

- ▶ When the **correct action** to take is not immediately obvious, an agent may need to **plan ahead**
- ▶ That means considering a **sequence of actions** that form a path to a **goal state**

Why searching?

Problem solving and search

- ▶ When the **correct action** to take is not immediately obvious, an agent may need to **plan ahead**
- ▶ That means considering a **sequence of actions** that form a path to a **goal state**
- ▶ Such an agent is called a **problem-solving agent**
- ▶ The computational process it undertakes is called **search**

Why searching?

Problem solving and search

- ▶ When the **correct action** to take is not immediately obvious, an agent may need to **plan ahead**
- ▶ That means considering a **sequence of actions** that form a path to a **goal state**
- ▶ Such an agent is called a **problem-solving agent**
- ▶ The computational process it undertakes is called **search**

This happens often in the real world, where there is a **cost** associated with our **actions**.

An agent on a touring vacation

Problem solving and search

Imagine an agent enjoying a touring vacation in Romania: it wants to take in the sights, improve its Romanian, enjoy the nightlife, avoid hangovers. . . **a complex decision problem!**

An agent on a touring vacation

Problem solving and search

Imagine an agent enjoying a touring vacation in Romania: it wants to take in the sights, improve its Romanian, enjoy the nightlife, avoid hangovers. . . a complex decision problem!

Suppose the agent is currently in **Arad**, with a nonrefundable ticket to fly out of **Bucharest** the following day.

An agent on a touring vacation

Problem solving and search

Imagine an agent enjoying a touring vacation in Romania: it wants to take in the sights, improve its Romanian, enjoy the nightlife, avoid hangovers. . . **a complex decision problem!**

Suppose the agent is currently in **Arad**, with a **nonrefundable ticket** to fly out of **Bucharest** the following day.

- ▶ It observes street signs: three roads lead out of Arad, to **Sibiu**, **Timisoara**, and **Zerind**
- ▶ None of these are the goal. . .
- ▶ Unless it knows the geography of Romania, **it won't know which road to follow!**

An agent on a touring vacation

Problem solving and search

Imagine an agent enjoying a touring vacation in Romania: it wants to take in the sights, improve its Romanian, enjoy the nightlife, avoid hangovers. . . **a complex decision problem!**

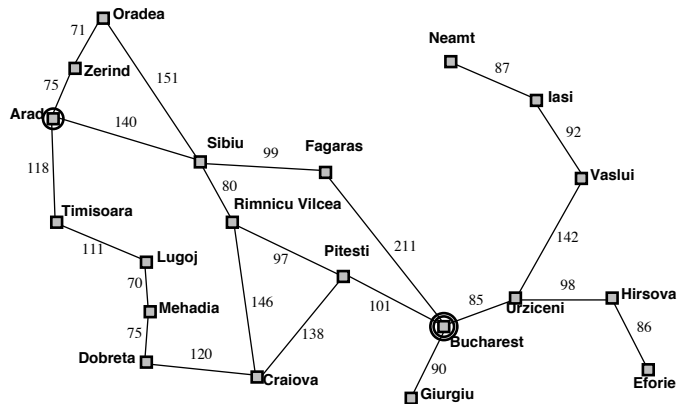
Suppose the agent is currently in **Arad**, with a **nonrefundable ticket** to fly out of **Bucharest** the following day.

- ▶ It observes street signs: three roads lead out of Arad, to **Sibiu**, **Timisoara**, and **Zerind**
- ▶ None of these are the goal. . .
- ▶ Unless it knows the geography of Romania, **it won't know which road to follow!**

With no additional information, the agent could only pick a road **at random**.
We will assume our agents have access to information about the world, e.g., a map.

An example about search

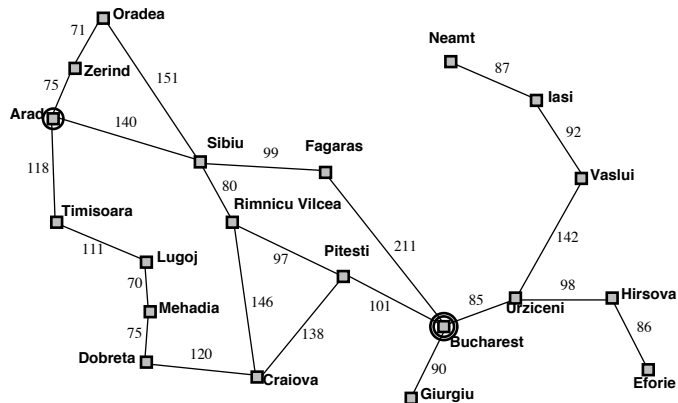
Problem solving and search



A simplified map of part of Romania, with road distances in miles.

An example about search

Problem solving and search



A simplified map of part of Romania, with road distances in miles.

Find a **sequence of cities** to drive through, from **Arad** to **Bucharest**.

How to solve it?

Problem solving and search

Our agent can follow a **four-phase problem-solving process**:

1. **Goal formulation**: adopt the goal of reaching Bucharest

How to solve it?

Problem solving and search

Our agent can follow a **four-phase problem-solving process**:

1. **Goal formulation**: adopt the goal of reaching Bucharest
2. **Problem formulation**: decide what other **states** there are, and what **actions** we can take

How to solve it?

Problem solving and search

Our agent can follow a **four-phase problem-solving process**:

1. **Goal formulation**: adopt the goal of reaching Bucharest
2. **Problem formulation**: decide what other **states** there are, and what **actions** we can take
3. **Search**: simulate sequences of actions to find one that reaches the goal

How to solve it?

Problem solving and search

Our agent can follow a **four-phase problem-solving process**:

1. **Goal formulation**: adopt the goal of reaching Bucharest
2. **Problem formulation**: decide what other **states** there are, and what **actions** we can take
3. **Search**: simulate sequences of actions to find one that reaches the goal
 - ▶ This sequence (or a subsequence) of actions is **the solution**!

How to solve it?

Problem solving and search

Our agent can follow a **four-phase problem-solving process**:

1. **Goal formulation**: adopt the goal of reaching Bucharest
2. **Problem formulation**: decide what other **states** there are, and what **actions** we can take
3. **Search**: simulate sequences of actions to find one that reaches the goal
 - This sequence (or a subsequence) of actions is **the solution**!
4. **Execution**: carry out the actions in the solution, one at a time

How to solve it?

Problem solving and search

We need to define the **state space**: all **possible states** of the environment.

How to solve it?

Problem solving and search

We need to define the **state space**: all **possible states** of the environment.

- **State space**: all locations in the Romania map

How to solve it?

Problem solving and search

We need to define the **state space**: all **possible states** of the environment.

- ▶ **State space**: all locations in the Romania map
- ▶ **Initial state**: *Arad*

How to solve it?

Problem solving and search

We need to define the **state space**: all **possible states** of the environment.

- ▶ **State space**: all locations in the Romania map
- ▶ **Initial state**: *Arad*
- ▶ **Goal state**: *Bucharest*

How to solve it?

Problem solving and search

We need to define the **state space**: all **possible states** of the environment.

- ▶ **State space**: all locations in the Romania map
- ▶ **Initial state**: *Arad*
- ▶ **Goal state**: *Bucharest*
- ▶ **Actions**: *Go from Arad to Sibiu, and to Timisoara, ...*

How to solve it?

Problem solving and search

We need to define the **state space**: all **possible states** of the environment.

- ▶ **State space**: all locations in the Romania map
- ▶ **Initial state**: *Arad*
- ▶ **Goal state**: *Bucharest*
- ▶ **Actions**: *Go from Arad to Sibiu, and to Timisoara, ...*
 - ▶ Each action leads to another state via the **transition model**, and has an **action cost**.

Make it formal with mathematics

Problem solving and search

The **state space** is a **set**:

$$S = \{Oradea, Zerind, Arad, Timisoara, Lugoj, Dobreta, \dots\}$$

Make it formal with mathematics

Problem solving and search

The **state space** is a **set**:

$$S = \{Oradea, Zerind, Arad, Timisoara, Lugoj, Dobreta, \dots\}$$

The **starting** s_0 and **goal** s_g states are **elements** of the **state space**: $s_0 = Arad$ and $s_g = Bucharest$

Make it formal with mathematics

Problem solving and search

The **state space** is a **set**:

$$S = \{Oradea, Zerind, Arad, Timisoara, Lugoj, Dobreta, \dots\}$$

The **starting** s_0 and **goal** s_g states are **elements** of the **state space**: $s_0 = Arad$ and $s_g = Bucharest$

The **transition model** can be summarised by a **function** *Result* which maps a **state** (origin) and an **action** to **another state** (destination): $Result : S \times A \rightarrow S$. E.g., $Result(Arad, ToSibiu) = Sibiu$

Make it formal with mathematics

Problem solving and search

The **state space** is a **set**:

$$S = \{Oradea, Zerind, Arad, Timisoara, Lugoj, Dobreta, \dots\}$$

The **starting** s_0 and **goal** s_g states are **elements** of the **state space**: $s_0 = Arad$ and $s_g = Bucharest$

The **transition model** can be summarised by a **function** *Result* which maps a **state** (origin) and an **action** to **another state** (destination): $Result : S \times A \rightarrow S$. E.g., $Result(Arad, ToSibiu) = Sibiu$

The **action cost function** can be another **function**, c , which maps **an action** (taken from a state to another) to a **cost** (a number): $c : S \times A \times S \rightarrow \mathbb{R}$

Make it formal with mathematics

Problem solving and search

The **state space** is a **set**:

$$S = \{Oradea, Zerind, Arad, Timisoara, Lugoj, Dobreta, \dots\}$$

The **starting** s_0 and **goal** s_g states are **elements** of the **state space**: $s_0 = Arad$ and $s_g = Bucharest$

The **transition model** can be summarised by a **function** *Result* which maps a **state** (origin) and an **action** to **another state** (destination): $Result : S \times A \rightarrow S$. E.g., $Result(Arad, ToSibiu) = Sibiu$

The **action cost function** can be another **function**, c , which maps **an action** (taken from a state to another) to a **cost** (a number): $c : S \times A \times S \rightarrow \mathbb{R}$

Together, the states and the transitions between them form a **state space graph**: **vertices** are states, and **directed edges** are actions. The Romania map you saw earlier is exactly this!

Make it formal with mathematics

Problem solving and search

The **state space** is a **set**:

$$S = \{Oradea, Zerind, Arad, Timisoara, Lugoj, Dobreta, \dots\}$$

The **starting** s_0 and **goal** s_g states are **elements** of the **state space**: $s_0 = Arad$ and $s_g = Bucharest$

The **transition model** can be summarised by a **function** *Result* which maps a **state** (origin) and an **action** to **another state** (destination): $Result : S \times A \rightarrow S$. E.g., $Result(Arad, ToSibiu) = Sibiu$

The **action cost function** can be another **function**, c , which maps **an action** (taken from a state to another) to a **cost** (a number): $c : S \times A \times S \rightarrow \mathbb{R}$

Together, the states and the transitions between them form a **state space graph**: **vertices** are states, and **directed edges** are actions. The Romania map you saw earlier is exactly this!

Having it in mathematical terms makes it **easier to code!**

Standardised vs. real-world problems

Problem solving and search

The problem-solving approach has been applied to a vast array of tasks. We distinguish between:

Standardised vs. real-world problems

Problem solving and search

The problem-solving approach has been applied to a vast array of tasks. We distinguish between:

- ▶ **Standardised problems:** illustrate or exercise problem-solving methods

Standardised vs. real-world problems

Problem solving and search

The problem-solving approach has been applied to a vast array of tasks. We distinguish between:

- ▶ **Standardised problems:** illustrate or exercise problem-solving methods
 - ▶ Example: *grid world*, sliding-tile puzzle, ...

Standardised vs. real-world problems

Problem solving and search

The problem-solving approach has been applied to a vast array of tasks. We distinguish between:

- ▶ **Standardised problems:** illustrate or exercise problem-solving methods
 - ▶ Example: *grid world*, sliding-tile puzzle, ...
- ▶ **Real-world problems:** solutions people actually use

Standardised vs. real-world problems

Problem solving and search

The problem-solving approach has been applied to a vast array of tasks. We distinguish between:

- ▶ **Standardised problems:** illustrate or exercise problem-solving methods
 - ▶ Example: *grid world*, sliding-tile puzzle, ...
- ▶ **Real-world problems:** solutions people actually use
 - ▶ Example: *route-finding*, robot navigation, VLSI layout, ...

Grid world problems

Problem solving and search

A **grid world** is a two-dimensional rectangular array of square cells.

Grid world problems

Problem solving and search

A **grid world** is a two-dimensional rectangular array of square cells.

- ▶ An agent can typically move to any **obstacle-free adjacent cell** (horizontally or vertically, sometimes diagonally)

Grid world problems

Problem solving and search

A **grid world** is a two-dimensional rectangular array of square cells.

- ▶ An agent can typically move to any **obstacle-free adjacent cell** (horizontally or vertically, sometimes diagonally)
- ▶ Cells can contain **objects**, which the agent can pick up, push, or otherwise act upon

Grid world problems

Problem solving and search

A **grid world** is a two-dimensional rectangular array of square cells.

- ▶ An agent can typically move to any **obstacle-free adjacent cell** (horizontally or vertically, sometimes diagonally)
- ▶ Cells can contain **objects**, which the agent can pick up, push, or otherwise act upon
- ▶ A **wall** or other impassable obstacle prevents the agent from moving into a cell

Grid world problems

Problem solving and search

A **grid world** is a two-dimensional rectangular array of square cells.

- ▶ An agent can typically move to any **obstacle-free adjacent cell** (horizontally or vertically, sometimes diagonally)
- ▶ Cells can contain **objects**, which the agent can pick up, push, or otherwise act upon
- ▶ A **wall** or other impassable obstacle prevents the agent from moving into a cell

Grid world problems

Problem solving and search

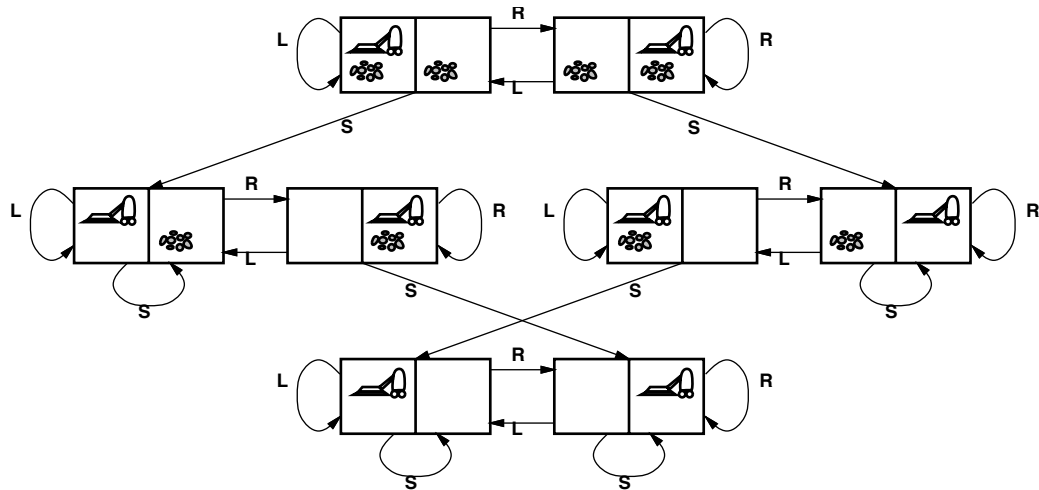
A **grid world** is a two-dimensional rectangular array of square cells.

- ▶ An agent can typically move to any **obstacle-free adjacent cell** (horizontally or vertically, sometimes diagonally)
- ▶ Cells can contain **objects**, which the agent can pick up, push, or otherwise act upon
- ▶ A **wall** or other impassable obstacle prevents the agent from moving into a cell

The **vacuum world** is a grid world problem!

Another example: Vacuum world

Problem solving and search



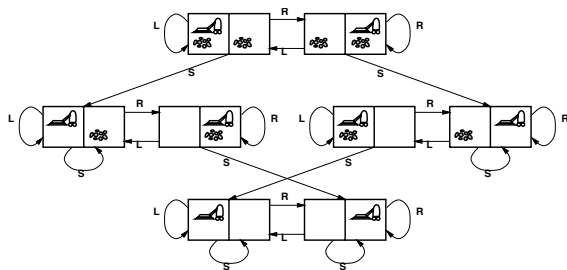
1

¹p. 85 in the textbook.

Another example: Vacuum world

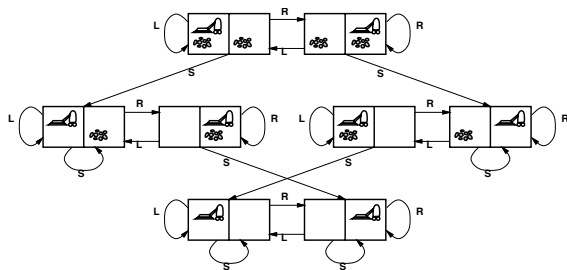
Problem solving and search

► Initial state: any state



Another example: Vacuum world

Problem solving and search

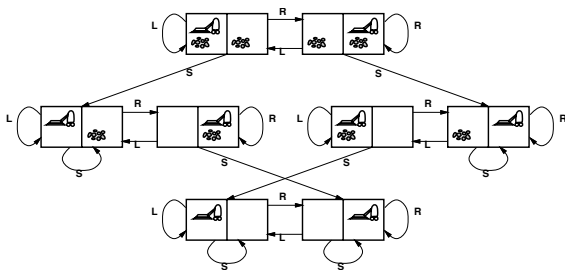


► Initial state: any state

► Goal states: no dirt

Another example: Vacuum world

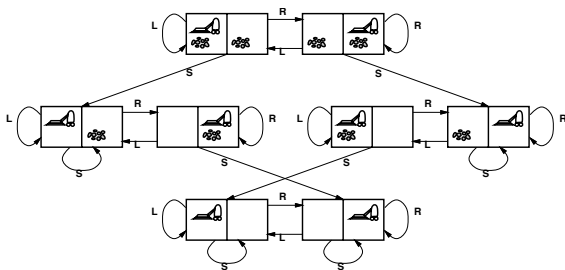
Problem solving and search



- Initial state: any state
- Goal states: no dirt
- Actions: *Suck, moveLeft, moveRight*
... or *Suck, Forward, Backward, TurnLeft, TurnRight, ...*

Another example: Vacuum world

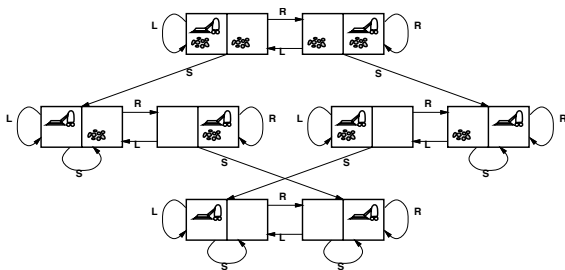
Problem solving and search



- ▶ Initial state: any state
- ▶ Goal states: no dirt
- ▶ Actions: *Suck*, *moveLeft*, *moveRight* ... or *Suck*, *Forward*, *Backward*, *TurnLeft*, *TurnRight*, ...
- ▶ Transition model: *Suck* cleans the location, *TurnLeft* changes the direction of agent 90 degrees

Another example: Vacuum world

Problem solving and search



- Initial state: any state
- Goal states: no dirt
- Actions: *Suck, moveLeft, moveRight* ... or *Suck, Forward, Backward, TurnLeft, TurnRight, ...*
- Transition model: *Suck* cleans the location, *TurnLeft* changes the direction of agent 90 degrees
- Action cost: 1 unit

The route-finding problem

Problem solving and search

We have already seen the **route-finding problem**: it is defined in terms of specified **locations** and **transitions** along edges between them.

The route-finding problem

Problem solving and search

We have already seen the **route-finding problem**: it is defined in terms of specified **locations** and **transitions** along edges between them.

Sounds familiar? The **Romania example** was a route-finding problem!

The route-finding problem

Problem solving and search

We have already seen the **route-finding problem**: it is defined in terms of specified **locations** and **transitions** along edges between them.

Sounds familiar? The **Romania example** was a route-finding problem!

- Route-finding algorithms are used in web sites and in-car navigation systems

The route-finding problem

Problem solving and search

We have already seen the **route-finding problem**: it is defined in terms of specified **locations** and **transitions** along edges between them.

Sounds familiar? The **Romania example** was a route-finding problem!

- ▶ Route-finding algorithms are used in web sites and in-car navigation systems
 - ▶ Main complications: varying costs (traffic delays), rerouting (road closures)

Applications

Problem solving and search

But the real world is usually **more complex!**

- ▶ Resources are limited (and **costs** become important!)

Applications

Problem solving and search

But the real world is usually **more complex!**

- ▶ Resources are limited (and **costs** become important!)
- ▶ We have **constraints** and **restrictions**

Applications

Problem solving and search

But the real world is usually **more complex!**

- ▶ Resources are limited (and **costs** become important!)
- ▶ We have **constraints** and **restrictions**
- ▶ We need to be quick and cannot freely **explore**

Applications

Problem solving and search

The **Travelling Salesperson Problem**: find shortest route visiting each location once and returns to initial location.

- For example: Delivery services



Applications

Problem solving and search

The **Travelling Salesperson Problem**: find shortest route visiting each location once and returns to initial location.

- For example: Delivery services
- The Holidays in Romania example



Applications

Problem solving and search

The **Travelling Salesperson Problem**: find shortest route visiting each location once and returns to initial location.

- ▶ For example: Delivery services (and you can always make it more complicated!)
 - ▶ Time windows
 - ▶ Closed roads
 - ▶ Traffic
- ▶ The Holidays in Romania example

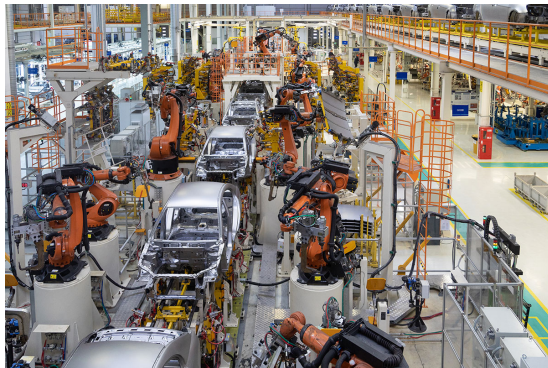


Applications

Problem solving and search

Assembly problems: find an order for assembling the parts of some object.

- ▶ For example: Manufacturing and design
(and you can always make it more complicated!)
 - ▶ Find the **optimal** order (minimum cost)
 - ▶ Reduce idle time on different machines
 - ▶ Assembly lines could be **dependent** on each other



Section 2

The search algorithm

What is a search algorithm?

Search algorithms

A **search algorithm** takes a search problem as input, and returns a **solution**, or an indication of **failure**.

What is a search algorithm?

Search algorithms

A **search algorithm** takes a search problem as input, and returns a **solution**, or an indication of **failure**.

To find that solution, we **superimpose** a **search tree** over the **state space graph** you already know.

What is a search algorithm?

Search algorithms

A **search algorithm** takes a search problem as input, and returns a **solution**, or an indication of **failure**.

To find that solution, we **superimpose** a **search tree** over the **state space graph** you already know.

- Each **node** in the search tree corresponds to a **state**; each edge corresponds to an **action**

What is a search algorithm?

Search algorithms

A **search algorithm** takes a search problem as input, and returns a **solution**, or an indication of **failure**.

To find that solution, we **superimpose** a **search tree** over the **state space graph** you already know.

- ▶ Each **node** in the search tree corresponds to a **state**; each edge corresponds to an **action**
- ▶ The **root** of the tree is the **initial state**

What is a search algorithm?

Search algorithms

A **search algorithm** takes a search problem as input, and returns a **solution**, or an indication of **failure**.

To find that solution, we **superimpose** a **search tree** over the **state space graph** you already know.

- ▶ Each **node** in the search tree corresponds to a **state**; each edge corresponds to an **action**
- ▶ The **root** of the tree is the **initial state**

What is a search algorithm?

Search algorithms

A **search algorithm** takes a search problem as input, and returns a **solution**, or an indication of **failure**.

To find that solution, we **superimpose** a **search tree** over the **state space graph** you already know.

- ▶ Each **node** in the search tree corresponds to a **state**; each edge corresponds to an **action**
- ▶ The **root** of the tree is the **initial state**

Let's see this in action, using our Romania example!

Exploring the state space

Search algorithms

- ▶ Most of the time, it is not feasible (or it is too expensive) to build and represent the entire state graph.

Exploring the state space

Search algorithms

- ▶ Most of the time, it is not feasible (or it is too expensive) to build and represent the entire state graph.
- ▶ The problem solver agent generates a solution by **incrementally exploring** a small portion of the graph

Exploring the state space

Search algorithms

- ▶ Most of the time, it is not feasible (or it is too expensive) to build and represent the entire state graph.
- ▶ The problem solver agent generates a solution by **incrementally exploring** a small portion of the graph
- ▶ We **simulate** the exploration by generating **successors** of **already-explored states**.

Exploring the state space

Search algorithms

- ▶ Most of the time, it is not feasible (or it is too expensive) to build and represent the entire state graph.
- ▶ The problem solver agent generates a solution by **incrementally exploring** a small portion of the graph
- ▶ We **simulate** the exploration by generating **successors** of **already-explored states**.

Exploring the state space

Search algorithms

- ▶ Most of the time, it is not feasible (or it is too expensive) to build and represent the entire state graph.
- ▶ The problem solver agent generates a solution by **incrementally exploring** a small portion of the graph
- ▶ We **simulate** the exploration by generating **successors** of **already-explored states**.

This process will be **repeated** until we either **find a solution**, or **fail** (by running out of time, of states, of resources. . .)

The search procedure

Search algorithms

You are standing on the initial node.

1. Check where you are standing: is it the goal?²

²Consider that the 'goal check' is dependent on the algorithm!

The search procedure

Search algorithms

You are standing on the initial node.

1. Check where you are standing: is it the **goal**?²
2. If not, what are the nodes to be explored from here?

²Consider that the 'goal check' is dependent on the algorithm!

The search procedure

Search algorithms

You are standing on the initial node.

1. Check where you are standing: is it the **goal**?²
2. If not, what are the nodes to be explored from here?
3. **Expand** the node you are in

²Consider that the 'goal check' is dependent on the algorithm!

The search procedure

Search algorithms

You are standing on the initial node.

1. Check where you are standing: is it the **goal**?²
2. If not, what are the nodes to be explored from here?
3. **Expand** the node you are in
4. Add the successor nodes into the **frontier**

²Consider that the 'goal check' is dependent on the algorithm!

The search procedure

Search algorithms

You are standing on the initial node.

1. Check where you are standing: is it the **goal**?²
2. If not, what are the nodes to be explored from here?
3. **Expand** the node you are in
4. Add the successor nodes into the **frontier**
5. Select (according to certain **criteria**—an evaluation function f —) the next node to expand, and move there

²Consider that the ‘goal check’ is dependent on the algorithm!

The search procedure

Search algorithms

You are standing on the initial node.

1. Check where you are standing: is it the **goal**?²
2. If not, what are the nodes to be explored from here?
3. **Expand** the node you are in
4. Add the successor nodes into the **frontier**
5. Select (according to certain **criteria**—an evaluation function f —) the next node to expand, and move there

²Consider that the ‘goal check’ is dependent on the algorithm!

The search procedure

Search algorithms

You are standing on the initial node.

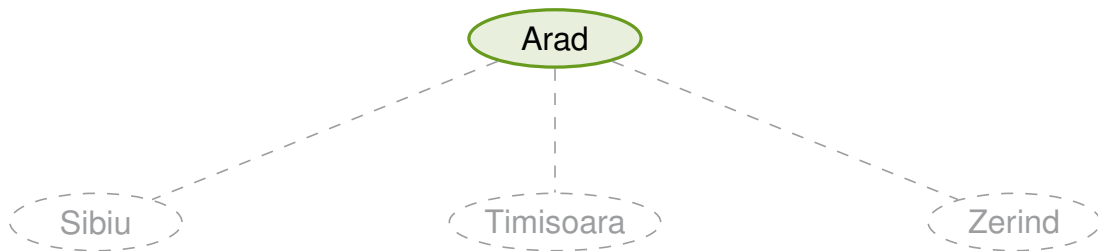
1. Check where you are standing: is it the **goal**?²
2. If not, what are the nodes to be explored from here?
3. **Expand** the node you are in
4. Add the successor nodes into the **frontier**
5. Select (according to certain **criteria**—an evaluation function f —) the next node to expand, and move there

And then **repeat**!

²Consider that the ‘goal check’ is dependent on the algorithm!

The search tree in action

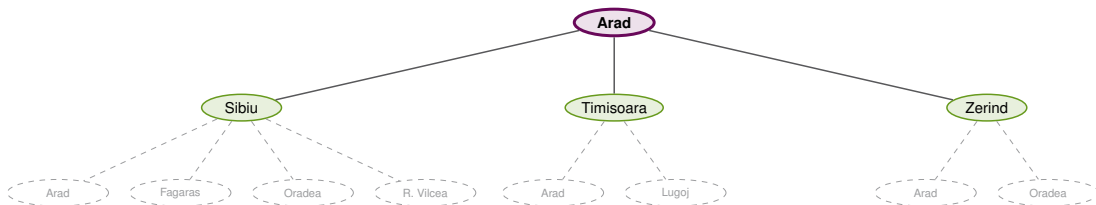
Search algorithms



The root of the search tree is the **initial state**: *Arad* is in the **frontier**.

The search tree in action

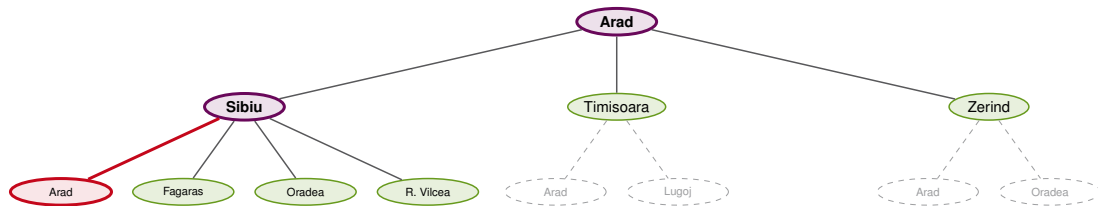
Search algorithms



We **expand** *Arad*: its 3 successors join the **frontier**.

The search tree in action

Search algorithms



We choose to expand *Sibiu* next. . . but one of its successors is *Arad* again!
That is a **cycle**: we already reached that state, so this path is not worth continuing.

What is a node?

Search algorithms

A **node** is a *representation* of a **state**. It is a data structure constituting a **part of a search tree**:

- ▶ The **state** of the node
- ▶ The **parent** of the node (or which node you came from)
- ▶ The **action** taken to reach the node
- ▶ The **path cost** of the search (so far)

Notice how a node is not a state, but **a step** in the search!

Terminology and the book I

Search algorithms

If a node is in the frontier, it does not mean it has been expanded! At least not for our book.

- ▶ The **frontier** are those nodes *I can expand*
- ▶ The set of **reached** nodes contains both the frontier AND the **expanded** nodes

So, formally, we know that

- ▶ $Frontier \subset Reached$, and
- ▶ $Frontier \cup Expanded = Reached$

And so, $Expanded = Reached \setminus Frontier$.

Terminology and the book II

Search algorithms

The book also uses *object-oriented programming* notation to refer to *pertaining* (or *belonging*):

- ▶ *node.STATE* is the *STATE* of *node*
- ▶ *node.PARENT* is the *PARENT* of *node*...

Operations are usually referred to as **functions**.

- ▶ *Search(problem)* is the *Search* procedure on the instance *problem*
- ▶ *IsEmpty(frontier)* is a function which returns true if the *frontier* is empty
- ▶ *Pop(frontier)* removes the top node of the frontier and returns it, while *Top(frontier)* just *peeks* at it (no removal)
- ▶ *Add(node, frontier)*...

You get the idea.

The frontier: how do we store it?

Search algorithms

We need a **data structure** to store the frontier. The natural choice is a **queue**.

The frontier: how do we store it?

Search algorithms

We need a **data structure** to store the frontier. The natural choice is a **queue**.

- ▶ **Priority queue**: pops the node with the **minimum cost** according to some evaluation function f

The frontier: how do we store it?

Search algorithms

We need a **data structure** to store the frontier. The natural choice is a **queue**.

- ▶ **Priority queue**: pops the node with the **minimum cost** according to some evaluation function f
- ▶ **FIFO queue** (First In, First Out): pops the node that was added **first**

The frontier: how do we store it?

Search algorithms

We need a **data structure** to store the frontier. The natural choice is a **queue**.

- ▶ **Priority queue**: pops the node with the **minimum cost** according to some evaluation function f
- ▶ **FIFO queue** (First In, First Out): pops the node that was added **first**
- ▶ **LIFO queue**, or **stack** (Last In, First Out): pops the **most recently added** node

The frontier: how do we store it?

Search algorithms

We need a **data structure** to store the frontier. The natural choice is a **queue**.

- ▶ **Priority queue**: pops the node with the **minimum cost** according to some evaluation function f
- ▶ **FIFO queue** (First In, First Out): pops the node that was added **first**
- ▶ **LIFO queue**, or **stack** (Last In, First Out): pops the **most recently added** node

The frontier: how do we store it?

Search algorithms

We need a **data structure** to store the frontier. The natural choice is a **queue**.

- ▶ **Priority queue**: pops the node with the **minimum cost** according to some evaluation function f
- ▶ **FIFO queue** (First In, First Out): pops the node that was added **first**
- ▶ **LIFO queue**, or **stack** (Last In, First Out): pops the **most recently added** node

Which one we use **depends on the algorithm!**

The frontier: how do we store it?

Search algorithms

After expanding *Arad*, the frontier holds *Sibiu*, *Timisoara*, and *Zerind* (added in that order). **Who pops next** depends entirely on the queue!

FIFO

Sibiu

Timisoara

Zerind

pops **Sibiu** (added first)

LIFO

Sibiu

Timisoara

Zerind

pops **Zerind** (added last)

Priority

Sibiu
140mi

Timisoara
118mi

Zerind
75mi

pops **Zerind** (cheapest: 75mi)

Notice: the stack and the priority queue happen to agree here — but for entirely different reasons!

One algorithm, many strategies

Search algorithms

All the strategies we will see are really the **same algorithm**: **Best-First-Search**.

³Recall: a node bundles a state, its parent, the action taken, and the path cost — not just a state!

One algorithm, many strategies

Search algorithms

All the strategies we will see are really the **same algorithm**: **Best-First-Search**.

- ▶ On each iteration, choose the node in the frontier with the **minimum value** of an **evaluation function** $f(n)$

³Recall: a node bundles a state, its parent, the action taken, and the path cost — not just a state!

One algorithm, many strategies

Search algorithms

All the strategies we will see are really the **same algorithm**: **Best-First-Search**.

- ▶ On each iteration, choose the node in the frontier with the **minimum value** of an **evaluation function** $f(n)$
 - ▶ Formally: f maps **nodes** to **numbers**, $f : Node \rightarrow \mathbb{R}$, and n is simply **a node** in the frontier³

³Recall: a node bundles a state, its parent, the action taken, and the path cost — not just a state!

One algorithm, many strategies

Search algorithms

All the strategies we will see are really the **same algorithm**: **Best-First-Search**.

- ▶ On each iteration, choose the node in the frontier with the **minimum value** of an **evaluation function** $f(n)$
 - ▶ Formally: f maps **nodes** to **numbers**, $f : Node \rightarrow \mathbb{R}$, and n is simply **a node** in the frontier³
- ▶ **Expand** it, generating its children

³Recall: a node bundles a state, its parent, the action taken, and the path cost — not just a state!

One algorithm, many strategies

Search algorithms

All the strategies we will see are really the **same algorithm**: **Best-First-Search**.

- ▶ On each iteration, choose the node in the frontier with the **minimum value** of an **evaluation function** $f(n)$
 - ▶ Formally: f maps **nodes** to **numbers**, $f : \text{Node} \rightarrow \mathbb{R}$, and n is simply **a node** in the frontier³
- ▶ **Expand** it, generating its children
- ▶ Repeat, until the goal is found (or the frontier is empty)

³Recall: a node bundles a state, its parent, the action taken, and the path cost — not just a state!

One algorithm, many strategies

Search algorithms

All the strategies we will see are really the **same algorithm**: **Best-First-Search**.

- ▶ On each iteration, choose the node in the frontier with the **minimum value** of an **evaluation function** $f(n)$
 - ▶ Formally: f maps **nodes** to **numbers**, $f : \text{Node} \rightarrow \mathbb{R}$, and n is simply **a node** in the frontier³
- ▶ **Expand** it, generating its children
- ▶ Repeat, until the goal is found (or the frontier is empty)

³Recall: a node bundles a state, its parent, the action taken, and the path cost — not just a state!

One algorithm, many strategies

Search algorithms

All the strategies we will see are really the **same algorithm**: **Best-First-Search**.

- ▶ On each iteration, choose the node in the frontier with the **minimum value** of an **evaluation function** $f(n)$
 - ▶ Formally: f maps **nodes** to **numbers**, $f : \text{Node} \rightarrow \mathbb{R}$, and n is simply **a node** in the frontier³
- ▶ **Expand** it, generating its children
- ▶ Repeat, until the goal is found (or the frontier is empty)

By choosing a **different** $f(n)$, we get a **different specific algorithm**. . . keep this in mind for the next slides!

³Recall: a node bundles a state, its parent, the action taken, and the path cost — not just a state!

Measuring problem-solving performance

Search algorithms

The **performance** of a search algorithm can be measured in different ways:

- ▶ **Completeness**: is the algorithm guaranteed to find a solution?
- ▶ **Cost optimality**: does it find a solution with the lowest path cost?
- ▶ **Time complexity**: how long does the algorithm take? (in seconds, operations, expanded states. . .)
- ▶ **Space complexity**: how much memory do we need, for example, in the *frontier* or *reached* sets?

Section 3

Uninformed search strategies

Uninformed vs. informed search

Uninformed search strategies

- ▶ **Uninformed** search: no clue about how close a state is to the goal

Uninformed vs. informed search

Uninformed search strategies

- ▶ **Uninformed** search: no clue about how close a state is to the goal
 - ▶ An agent in Arad has no idea whether Zerind or Sibiu is a better first step

Uninformed vs. informed search

Uninformed search strategies

- ▶ **Uninformed** search: no clue about how close a state is to the goal
 - ▶ An agent in Arad has no idea whether Zerind or Sibiu is a better first step
- ▶ **Informed** search: uses a **domain-specific hint** about the location of the goal

Uninformed vs. informed search

Uninformed search strategies

- ▶ **Uninformed** search: no clue about how close a state is to the goal
 - ▶ An agent in Arad has no idea whether Zerind or Sibiu is a better first step
- ▶ **Informed** search: uses a **domain-specific hint** about the location of the goal
 - ▶ An informed agent knows Sibiu is much closer to Bucharest than Zerind is

Uninformed vs. informed search

Uninformed search strategies

- ▶ **Uninformed** search: no clue about how close a state is to the goal
 - ▶ An agent in Arad has no idea whether Zerind or Sibiu is a better first step
- ▶ **Informed** search: uses a **domain-specific hint** about the location of the goal
 - ▶ An informed agent knows Sibiu is much closer to Bucharest than Zerind is

Uninformed vs. informed search

Uninformed search strategies

- ▶ **Uninformed** search: no clue about how close a state is to the goal
 - ▶ An agent in Arad has no idea whether Zerind or Sibiu is a better first step
- ▶ **Informed** search: uses a **domain-specific hint** about the location of the goal
 - ▶ An informed agent knows Sibiu is much closer to Bucharest than Zerind is

That hint will guide the search towards the goal, **more efficiently**. We will get there later in this lecture!

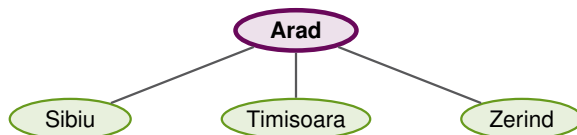
Searching with no information

Uninformed search strategies

Recall the fifth step in the [searching procedure](#) a few slides back:

The searching procedure revised

... 5. Select (according to certain **criteria**—a function f —) the next node to expand and move.



$f(n) = ?$ — f decides which one pops next!

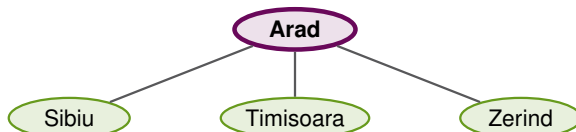
Searching with no information

Uninformed search strategies

Recall the fifth step in the [searching procedure](#) a few slides back:

The searching procedure revised

... 5. Select (according to certain **criteria**—a function f —) the next node to expand and move.



$f(n) = ?$ — f decides which one pops next!

► Breadth-first search (BFS)

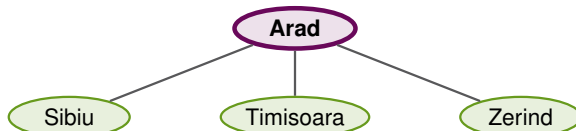
Searching with no information

Uninformed search strategies

Recall the fifth step in the [searching procedure](#) a few slides back:

The searching procedure revised

... 5. Select (according to certain **criteria**—a function f —) the next node to expand and move.



$f(n) = ?$ — f decides which one pops next!

- Breadth-first search (BFS)
- Depth-first search (DFS)

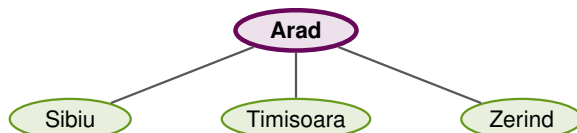
Searching with no information

Uninformed search strategies

Recall the fifth step in the [searching procedure](#) a few slides back:

The searching procedure revised

... 5. Select (according to certain **criteria**—a function f —) the next node to expand and move.



$f(n) = ?$ — f decides which one pops next!

- Breadth-first search (BFS)
- Depth-first search (DFS)
- Depth-limited search

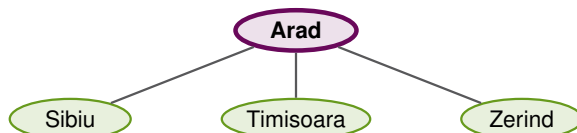
Searching with no information

Uninformed search strategies

Recall the fifth step in the [searching procedure](#) a few slides back:

The searching procedure revised

... 5. Select (according to certain **criteria**—a function f —) the next node to expand and move.



$f(n) = ?$ — f decides which one pops next!

- ▶ Breadth-first search (BFS)
- ▶ Depth-first search (DFS)
- ▶ Depth-limited search
- ▶ Iterative deepening

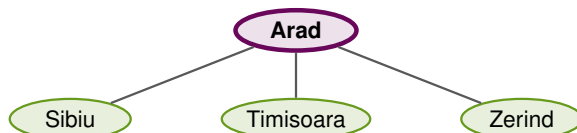
Searching with no information

Uninformed search strategies

Recall the fifth step in the [searching procedure](#) a few slides back:

The searching procedure revised

... 5. Select (according to certain **criteria**—a function f —) the next node to expand and move.



$f(n) = ?$ — f decides which one pops next!

- ▶ Breadth-first search (BFS)
- ▶ Depth-first search (DFS)
- ▶ Depth-limited search
- ▶ Iterative deepening
- ▶ Uniform-cost (Dijkstra)

Things to look out for

Search strategies

Implementation details vary a lot, and can be tricky!

- Is the algorithm checking for **redundant paths**⁴ (**graph search**) or not (**tree-like search**)?

⁴A **redundant path** is a worse way to reach an already-reached state; a **cycle** is one that loops back to an ancestor.

Things to look out for

Search strategies

Implementation details vary a lot, and can be tricky!

- ▶ Is the algorithm checking for **redundant paths**⁴ (**graph search**) or not (**tree-like search**)?
- ▶ Is the goal check performed **early** (when a node is *generated*) or **late** (when a node is *expanded*)?

⁴A **redundant path** is a worse way to reach an already-reached state; a **cycle** is one that loops back to an ancestor.

Things to look out for

Search strategies

Implementation details vary a lot, and can be tricky!

- ▶ Is the algorithm checking for **redundant paths**⁴ (**graph search**) or not (**tree-like search**)?
- ▶ Is the goal check performed **early** (when a node is *generated*) or **late** (when a node is *expanded*)?
- ▶ Is the algorithm **storing** all **reached states**, or **reconstructing the path** from a chain of parent nodes?

⁴A **redundant path** is a worse way to reach an already-reached state; a **cycle** is one that loops back to an ancestor.

Things to look out for

Search strategies

Implementation details vary a lot, and can be tricky!

- ▶ Is the algorithm checking for **redundant paths**⁴ (**graph search**) or not (**tree-like search**)?
- ▶ Is the goal check performed **early** (when a node is *generated*) or **late** (when a node is *expanded*)?
- ▶ Is the algorithm **storing** all **reached states**, or **reconstructing the path** from a chain of parent nodes?

Read the book!

To become familiar with the algorithms and their implementations details, you should read the book. These slides are not a replacement for the book; they are a summary of the most important points.

⁴A **redundant path** is a worse way to reach an already-reached state; a **cycle** is one that loops back to an ancestor.

Breadth First Search

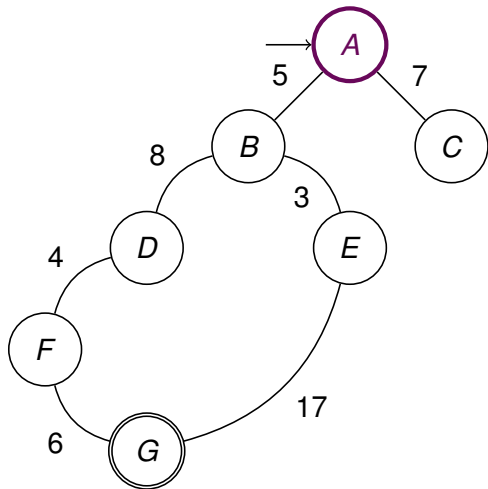
Uninformed search strategies

BFS prioritises old nodes first, and newly discovered ones last (hence the name, as it explores by *breadth* first)

- ▶ The *frontier* is a queue, i.e., “First In, First Out” (FIFO).
- ▶ Start at A and Goal is G.

1. Add A to frontier.
2. A is not goal, so *Expand*(A).
3. Add successors to *frontier*:
 - ▶ Is any of those the goal?^a No, so $frontier = \langle B, C \rangle$
4. Choose first element in *frontier*: B.

^ain BFS, we check early for the goal!



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Breadth First Search

Uninformed search strategies

BFS prioritises old nodes first, and newly discovered ones last (hence the name, as it explores by *breadth* first)

► The *frontier* is a queue, i.e., “First In, First Out” (FIFO).

► Start at A and Goal is G.

1. Add A to frontier.

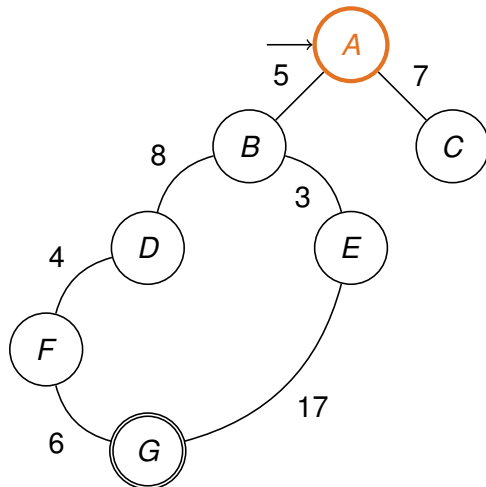
2. A is not goal, so *Expand(A)*.

3. Add successors to *frontier*:

► Is any of those the goal?^a No, so
frontier = $\langle B, C \rangle$

4. Choose *first element* in *frontier*: B.

^ain BFS, we check early for the goal!



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Breadth First Search

Uninformed search strategies

BFS prioritises old nodes first, and newly discovered ones last (hence the name, as it explores by *breadth* first)

► The *frontier* is a queue, i.e., “First In, First Out” (FIFO).

► Start at A and Goal is G.

1. Add A to frontier.

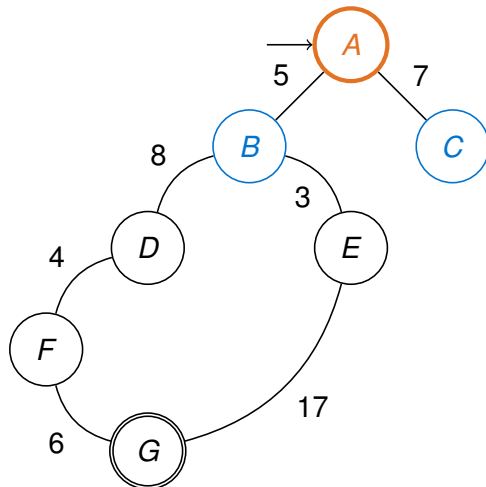
2. A is not goal, so *Expand*(A).

3. Add successors to frontier:

► Is any of those the goal?^a No, so
frontier = $\langle B, C \rangle$

4. Choose first element in frontier: B.

^ain BFS, we check early for the goal!



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Breadth First Search

Uninformed search strategies

BFS prioritises old nodes first, and newly discovered ones last (hence the name, as it explores by *breadth* first)

► The *frontier* is a queue, i.e., “First In, First Out” (FIFO).

► Start at A and Goal is G.

1. Add A to frontier.

2. A is not goal, so *Expand*(A).

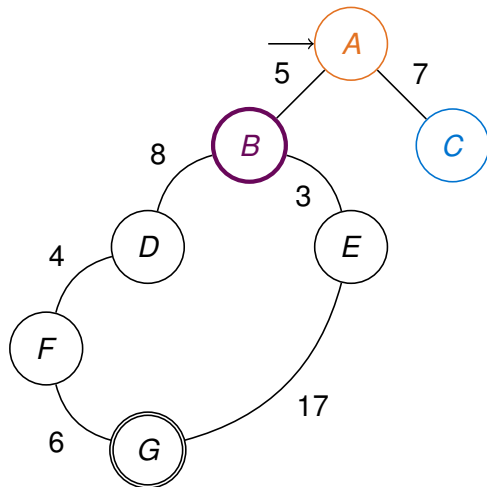
3. Add successors to *frontier*:

► Is any of those the goal?^a No, so $frontier = \langle B, C \rangle$

4. Choose first element in *frontier*: *B*.

And repeat. . .

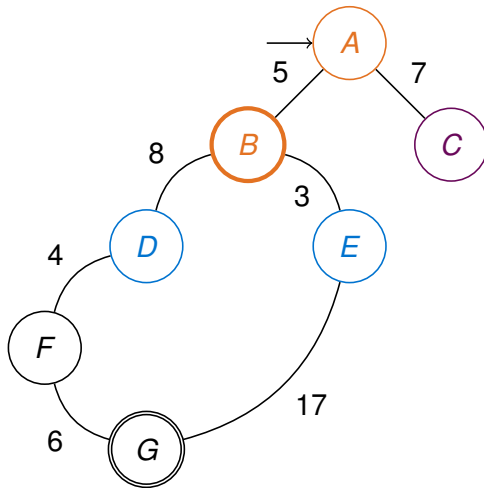
^ain BFS, we check early for the goal!



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Breadth First Search

Uninformed search strategies



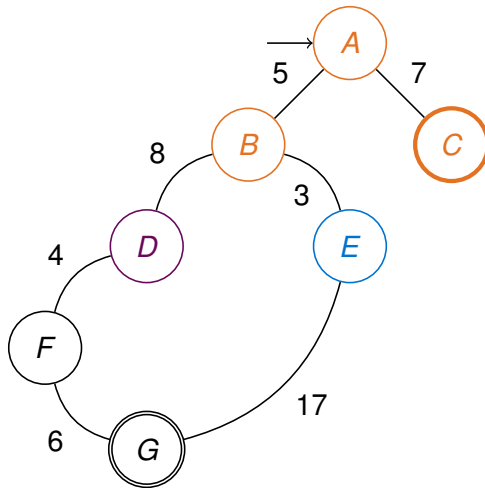
B expanded: *D* and *E* join the frontier. *C*, added earlier, is still first in line: expand *C* next.

frontier = $\langle C, D, E \rangle$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Breadth First Search

Uninformed search strategies



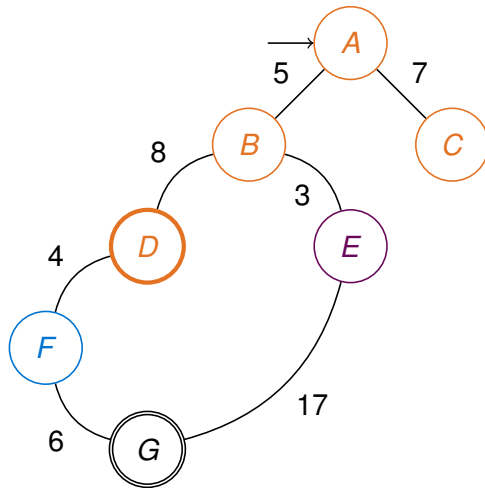
C is a dead end — no successors. D , added first among what's left, is next: expand D next.

$frontier = \langle D, E \rangle$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Breadth First Search

Uninformed search strategies



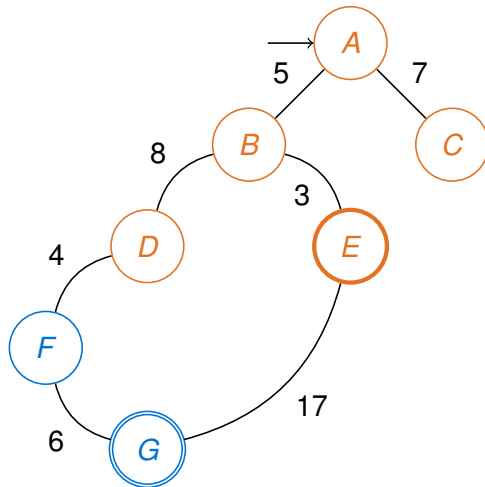
D expanded: *F* joins the frontier. *E*, added earlier, is still next in line: expand *E* next.

frontier = $\langle E, F \rangle$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Breadth First Search

Uninformed search strategies



E expanded: *G* is generated — checked **early**, so we stop right here!
frontier would become $\langle F, G \rangle$ — but we return before adding it!

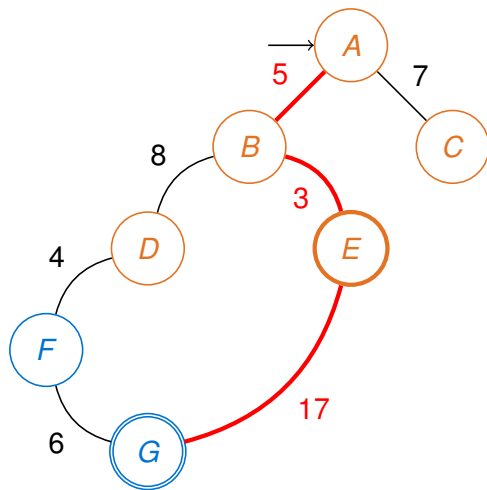
● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Breadth First Search

Uninformed search strategies

- ▶ We have **seen** the goal!^a
- ▶ We can reconstruct the solution by creating a *chain of parents* from the **goal**

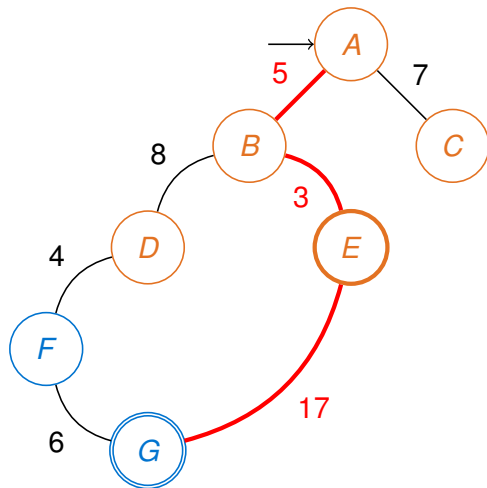
^aRemember we check for goal when adding to the frontier in BFS!



Breadth First Search

Uninformed search strategies

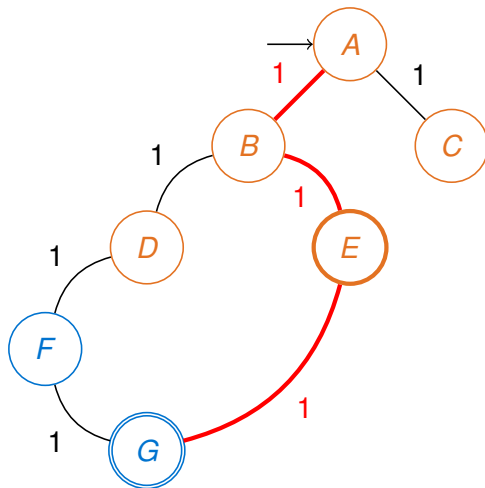
- **Not cost optimal**, unless all costs were equal!



Breadth First Search

Uninformed search strategies

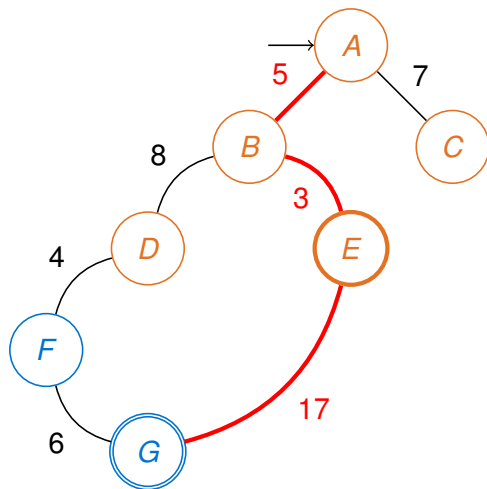
- **Not cost optimal**, unless all costs were equal!
- Like so!



Breadth First Search

Uninformed search strategies

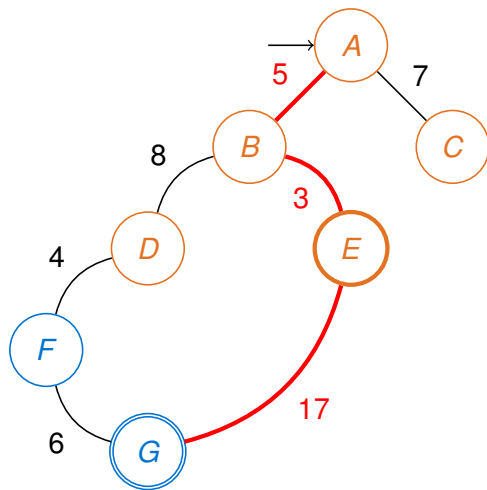
- ▶ **Not cost optimal**, unless all costs were equal!
- ▶ Like so!
- ▶ **Complete**: always finds a solution if space state is finite



Breadth First Search

Uninformed search strategies

- ▶ **Not cost optimal**, unless all costs were equal!
- ▶ Like so!
- ▶ **Complete**: always finds a solution if space state is finite
- ▶ Time and space complexity is insane $\mathcal{O}(b^d)$ where b is the branching factor (number of successors to consider) and d is the depth of the shallowest solution.



This was a summarised, but still quite detailed explanation. The following algorithms will be **summarised a bit more**, so

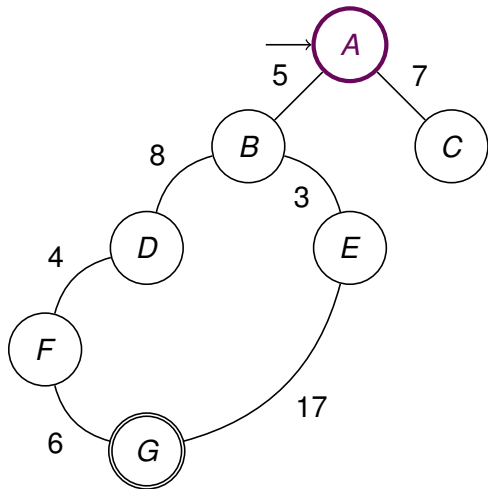
check your book for the step by step strategies.

Depth First Search

Uninformed search strategies

DFS prioritises **new nodes first**, and previously discovered ones go last (hence the name, as it explores by *depth* first)

- ▶ The *frontier* is a **stack**, i.e., “Last In, First Out” (LIFO)
 - ▶ Start at A and Goal is G, as before
1. Add A to frontier.
 2. A is not goal, so *Expand*(A).
 3. Add successors to *frontier* (in reverse order):
 - ▶ $frontier = \langle B, C \rangle$ — frontier grows towards the left!
 4. Choose **first element** in *frontier*: B.



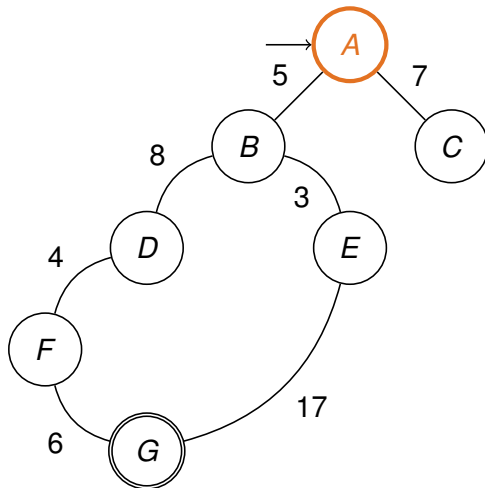
● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Depth First Search

Uninformed search strategies

DFS prioritises **new nodes first**, and previously discovered ones go last (hence the name, as it explores by *depth* first)

- ▶ The *frontier* is a **stack**, i.e., “Last In, First Out” (LIFO)
- ▶ Start at A and Goal is G, as before
- 1. Add A to frontier.
- 2. A is not goal, so *Expand(A)*.
- 3. Add successors to *frontier* (in reverse order):
 - ▶ $frontier = \langle B, C \rangle$ — frontier grows towards the left!
- 4. Choose **first element** in *frontier*: B.



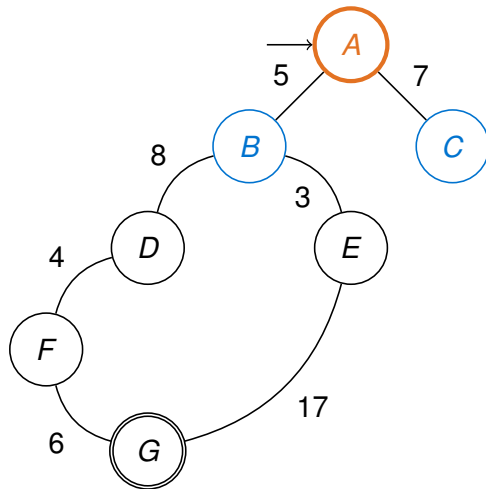
● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Depth First Search

Uninformed search strategies

DFS prioritises **new nodes first**, and previously discovered ones go last (hence the name, as it explores by *depth* first)

- ▶ The *frontier* is a **stack**, i.e., “Last In, First Out” (LIFO)
 - ▶ Start at A and Goal is G, as before
1. Add A to frontier.
 2. A is not goal, so *Expand*(A).
 3. Add successors to *frontier* (in reverse order):
 - ▶ *frontier* = $\langle B, C \rangle$ — frontier grows towards the left!
 4. Choose **first element** in *frontier*: B.



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

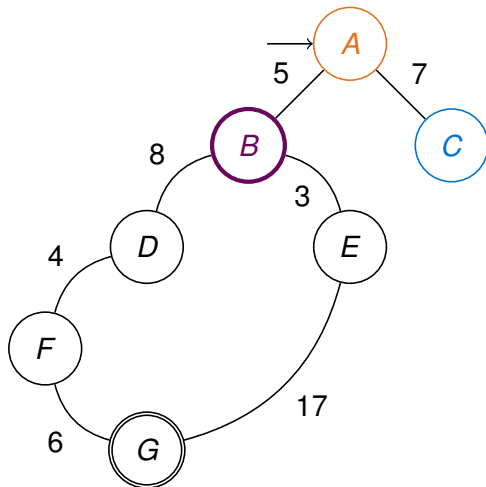
Depth First Search

Uninformed search strategies

DFS prioritises **new nodes first**, and previously discovered ones go last (hence the name, as it explores by *depth* first)

- ▶ The *frontier* is a **stack**, i.e., “Last In, First Out” (LIFO)
 - ▶ Start at A and Goal is G, as before
1. Add A to frontier.
 2. A is not goal, so *Expand(A)*.
 3. Add successors to *frontier* (in reverse order):
 - ▶ $frontier = \langle B, C \rangle$ — frontier grows towards the left!
 4. Choose **first element** in *frontier*: **B**.

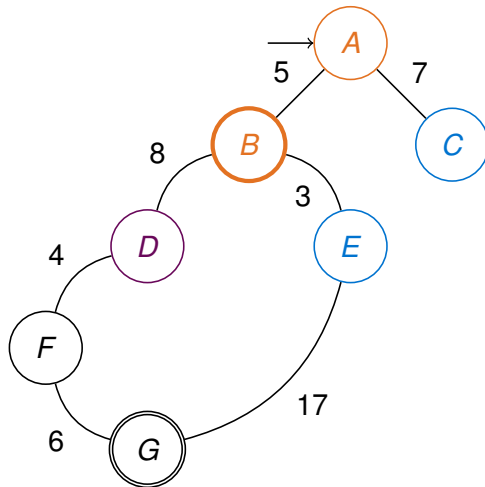
And repeat. . .



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Depth First Search

Uninformed search strategies



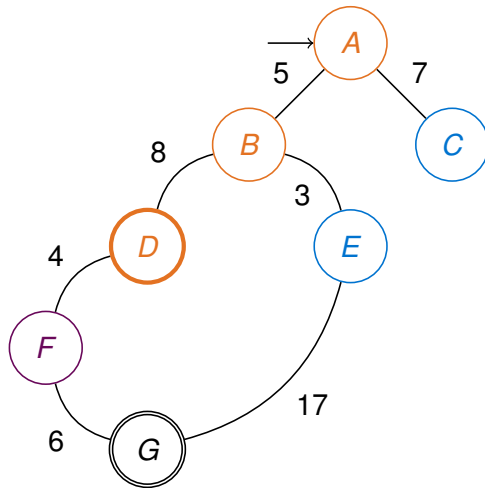
B expanded: *D* and *E* are pushed onto the stack, on top of *C*. *D* ends up on top: expand *D* next.

frontier = $\langle D, E, C \rangle$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Depth First Search

Uninformed search strategies



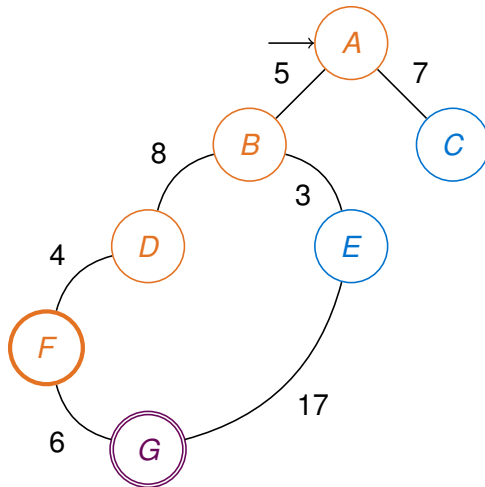
D expanded: *F* is pushed on top. *E* and *C* are still waiting further down the stack: expand *F* next.

frontier = $\langle F, E, C \rangle$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Depth First Search

Uninformed search strategies



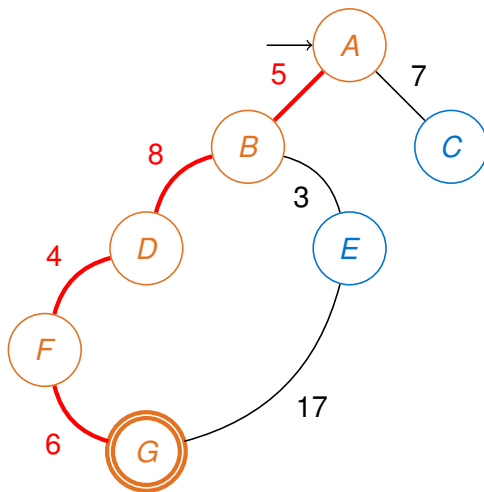
F expanded: *G* is pushed on top — and it's the goal! *E* and *C* were never even visited.

frontier = $\langle G, E, C \rangle$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Depth First Search

Uninformed search strategies



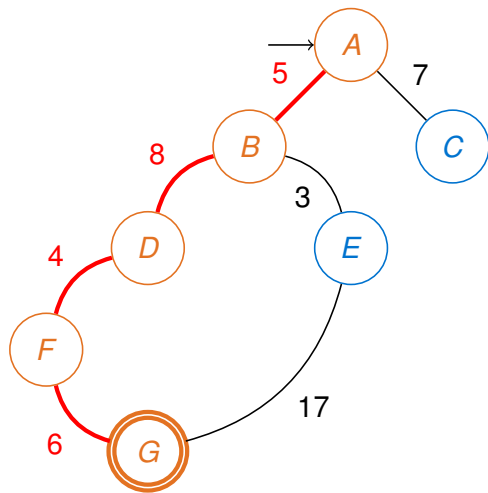
G expanded: it's the goal! Path: $A \rightarrow B \rightarrow D \rightarrow F \rightarrow G$, cost 23.

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Depth First Search

Uninformed search strategies

- **Not always cost optimal**

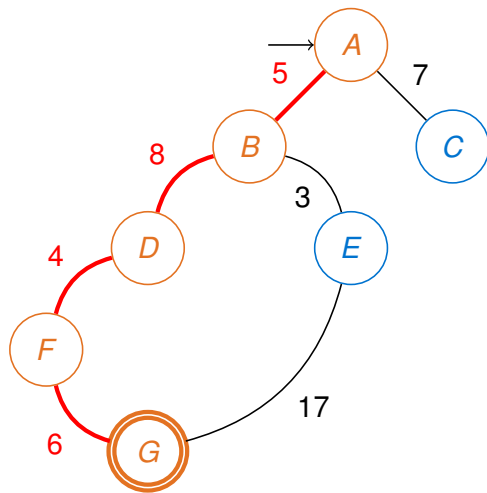


^abecause it is usually implemented as tree-like search

Depth First Search

Uninformed search strategies

- **Not always cost optimal**
- It returns the first solution found. We were lucky!

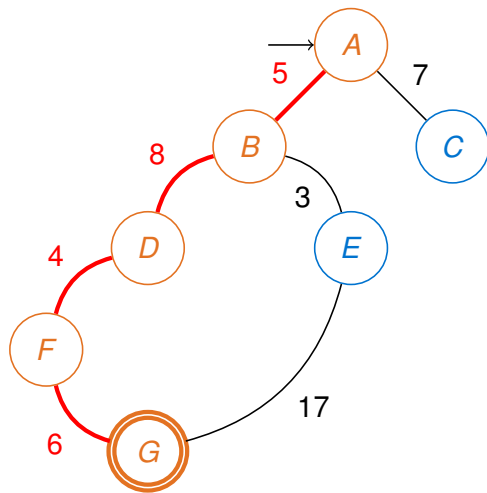


^abecause it is usually implemented as tree-like search

Depth First Search

Uninformed search strategies

- ▶ **Not always cost optimal**
- ▶ It returns the first solution found. We were lucky!
- ▶ **Not Complete:** fails in infinitely-deep spaces and spaces with loops^a

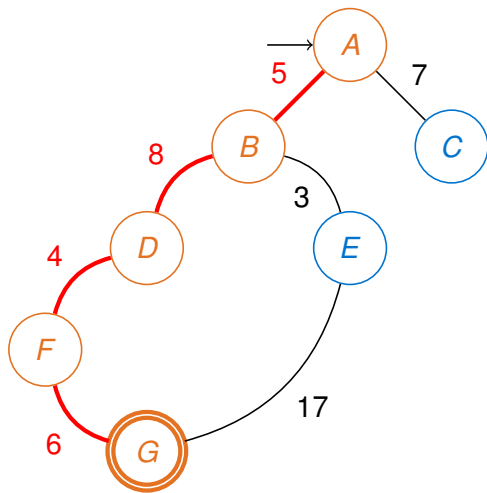


^abecause it is usually implemented as tree-like search

Depth First Search

Uninformed search strategies

- ▶ **Not always cost optimal**
- ▶ It returns the first solution found. We were lucky!
- ▶ **Not Complete:** fails in infinitely-deep spaces and spaces with loops^a
- ▶ Time complexity $\mathcal{O}(b^m)$, and space complexity is linear $\mathcal{O}(bm)$ where b is the branching factor and m is the maximum depth in the state space (tree version)

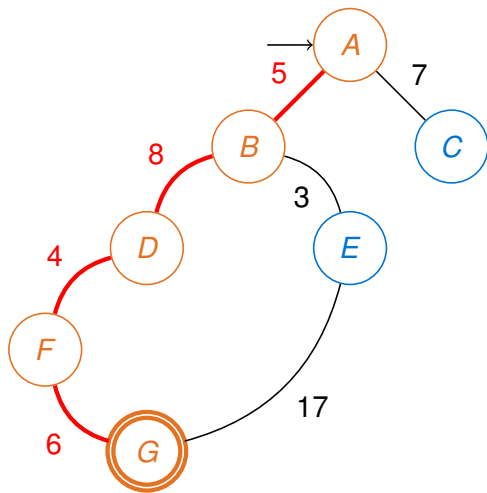


^abecause it is usually implemented as tree-like search

Depth First Search

Uninformed search strategies

- ▶ **Not always cost optimal**
- ▶ It returns the first solution found. We were lucky!
- ▶ **Not Complete:** fails in infinitely-deep spaces and spaces with loops^a
- ▶ Time complexity $\mathcal{O}(b^m)$, and space complexity is linear $\mathcal{O}(bm)$ where b is the branching factor and m is the maximum depth in the state space (tree version)
- ▶ One can make a smarter version of DFS with graph search (memory). Space complexity grows to exponential, and might still miss if on infinite spaces.



^abecause it is usually implemented as tree-like search

Depth-limited and Iterative deepening search

Uninformed search strategies

Two other ideas lie on imposing a limit on DFS, both as *tree-like search strategies*.

- ▶ **Depth-limited search:** DFS with a supplied **depth limit** ℓ — nodes at depth ℓ are treated as if they had no successors
- ▶ **Iterative deepening:** try $\ell = 0, 1, 2, \dots$ *iteratively*, until a solution is found or a set *cutoff*

Depth-limited and Iterative deepening search

Uninformed search strategies

Two other ideas lie on imposing a limit on DFS, both as *tree-like search strategies*.

- ▶ **Depth-limited search:** DFS with a supplied **depth limit** ℓ — nodes at depth ℓ are treated as if they had no successors
- ▶ **Iterative deepening:** try $\ell = 0, 1, 2, \dots$ *iteratively*, until a solution is found or a set *cutoff*
- ▶ Iterative deepening will try multiple levels and return either a **solution** if it exists, a **failure** if it does not, or a **cutoff**.
- ▶ A *cutoff* means the maximum depth we set previously was reached, so a solution might exist deeper than the levels we explored.

Depth-limited and Iterative deepening search

Uninformed search strategies

Two other ideas lie on imposing a limit on DFS, both as *tree-like search strategies*.

- ▶ **Depth-limited search**: DFS with a supplied **depth limit** ℓ — nodes at depth ℓ are treated as if they had no successors
- ▶ **Iterative deepening**: try $\ell = 0, 1, 2, \dots$ *iteratively*, until a solution is found or a set *cutoff*
- ▶ Iterative deepening will try multiple levels and return either a **solution** if it exists, a **failure** if it does not, or a **cutoff**.
- ▶ A *cutoff* means the maximum depth we set previously was reached, so a solution might exist deeper than the levels we explored.
- ▶ **Always complete** if solution exists and state space is finite
- ▶ **Not cost optimal** unless costs are the same (like BFS)
- ▶ Time complexity: $\mathcal{O}(b^d)$
- ▶ Space complexity: $\mathcal{O}(bd)$ (like DFS)

Slightly better than both DFS and BFS!

Uniform-Cost Search (Dijkstra)

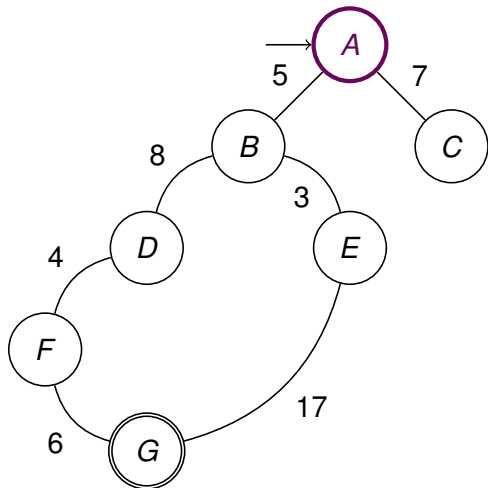
Uninformed search strategies

UCS always expands the node with the lowest **path cost** $g(n)$ so far: $f(n) = g(n)$.

- ▶ The *frontier* is a **priority queue**, ordered by $g(n)$.
- ▶ Goal check happens **late**^a (only when a node is *expanded*).
- ▶ Start at A and Goal is G, as before.

1. Add A ($g = 0$) to frontier.
2. A is not goal, so *Expand*(A).
3. Add successors: B ($g = 5$), C ($g = 7$).
4. Choose the **cheapest** node in *frontier*: B.

^aUnlike BFS, which checks early!



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Uniform-Cost Search (Dijkstra)

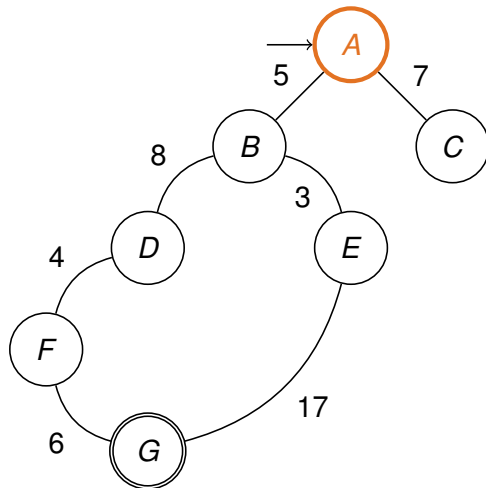
Uninformed search strategies

UCS always expands the node with the lowest **path cost** $g(n)$ so far: $f(n) = g(n)$.

- ▶ The *frontier* is a **priority queue**, ordered by $g(n)$.
- ▶ Goal check happens **late**^a (only when a node is *expanded*).
- ▶ Start at A and Goal is G, as before.

1. Add A ($g = 0$) to frontier.
2. A is not goal, so *Expand(A)*.
3. Add successors: B ($g = 5$), C ($g = 7$).
4. Choose the **cheapest** node in *frontier*: B.

^aUnlike BFS, which checks early!



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Uniform-Cost Search (Dijkstra)

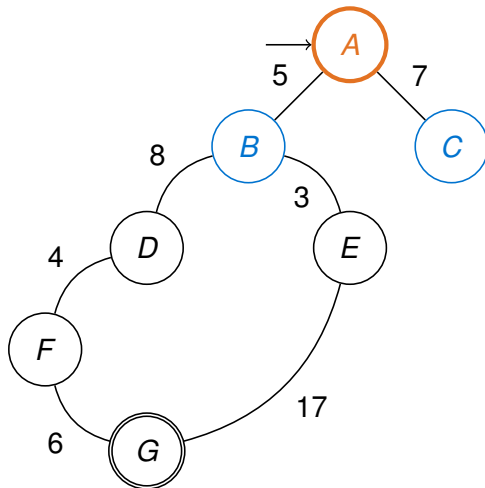
Uninformed search strategies

UCS always expands the node with the lowest **path cost** $g(n)$ so far: $f(n) = g(n)$.

- ▶ The *frontier* is a **priority queue**, ordered by $g(n)$.
- ▶ Goal check happens **late**^a (only when a node is *expanded*).
- ▶ Start at A and Goal is G, as before.

1. Add A ($g = 0$) to frontier.
2. A is not goal, so *Expand*(A).
3. Add successors: B ($g = 5$), C ($g = 7$).
4. Choose the **cheapest** node in *frontier*: B.

^aUnlike BFS, which checks early!



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Uniform-Cost Search (Dijkstra)

Uninformed search strategies

UCS always expands the node with the lowest **path cost** $g(n)$ so far: $f(n) = g(n)$.

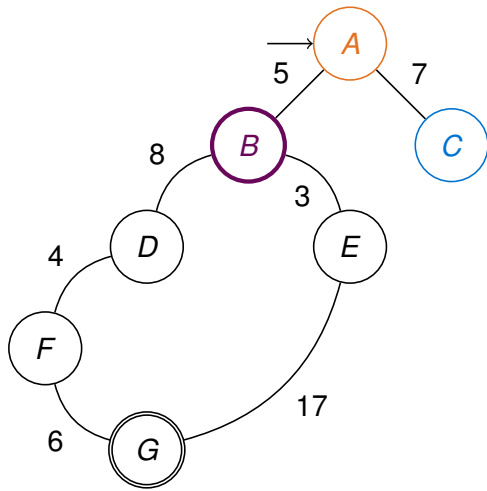
- ▶ The *frontier* is a **priority queue**, ordered by $g(n)$.
- ▶ Goal check happens **late**^a (only when a node is *expanded*).
- ▶ Start at A and Goal is G, as before.

1. Add A ($g = 0$) to frontier.
2. A is not goal, so *Expand*(A).
3. Add successors: B ($g = 5$), C ($g = 7$).
4. Choose the cheapest node in frontier: B.

And repeat...

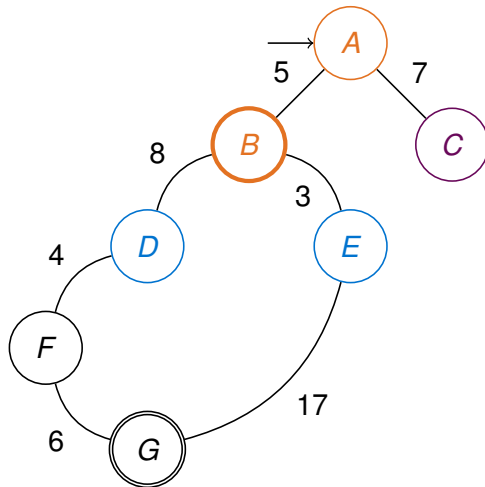
^aUnlike BFS, which checks early!

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached



Uniform-Cost Search (Dijkstra)

Uninformed search strategies

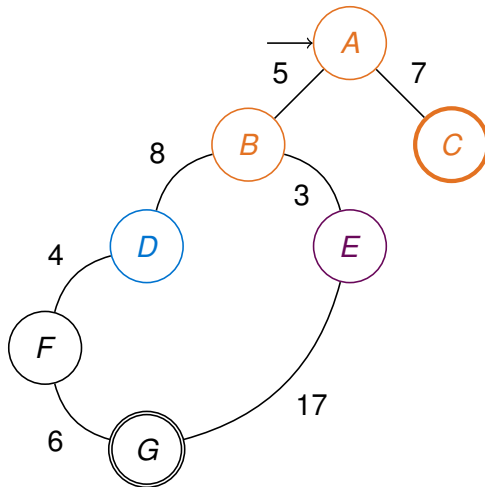


B expanded: *D*, *E* join the frontier. *C* is still cheaper: expand *C* next.
frontier = {*C*:7, *D*:13, *E*:8}

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Uniform-Cost Search (Dijkstra)

Uninformed search strategies



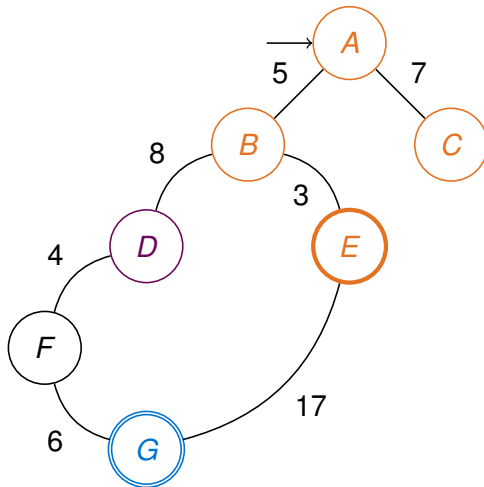
C is a dead end. Cheapest left is *E*: expand *E* next.

frontier = {*D*:13, *E*:8}

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Uniform-Cost Search (Dijkstra)

Uninformed search strategies

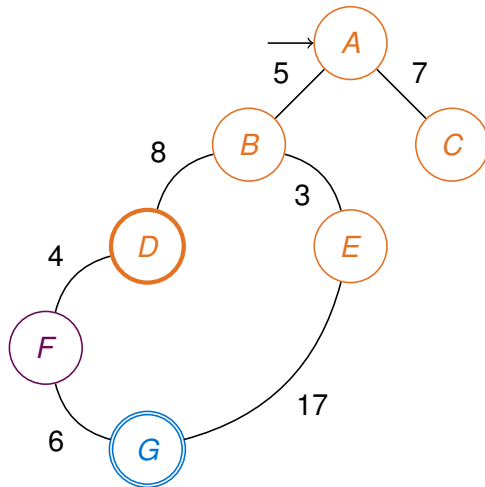


E expanded: *G* reached at $g=25$. . . not cheapest, so we do **not** stop! Expand *D*.
frontier = {*D*:13, *G*:25}

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Uniform-Cost Search (Dijkstra)

Uninformed search strategies

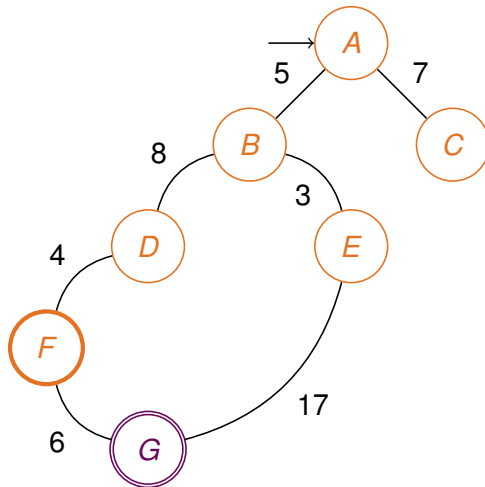


D expanded: *F* joins, cheaper than the *G* we already reached: expand *F* next.
frontier = {*F*:17, *G*:25}

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Uniform-Cost Search (Dijkstra)

Uninformed search strategies



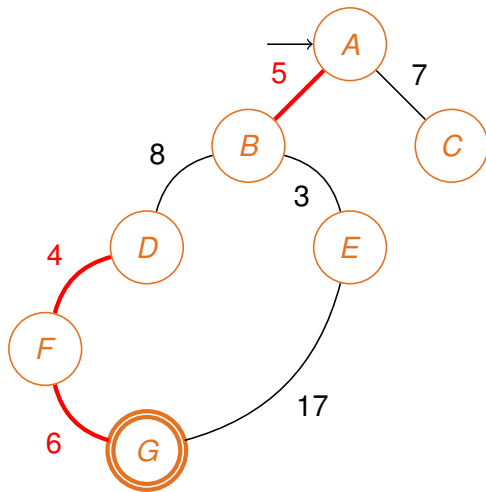
F expanded: a **cheaper path** to *G* found, $g=23 < 25$! It **replaces** the old one.
frontier = { *G*:23 }

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Uniform-Cost Search (Dijkstra)

Uninformed search strategies

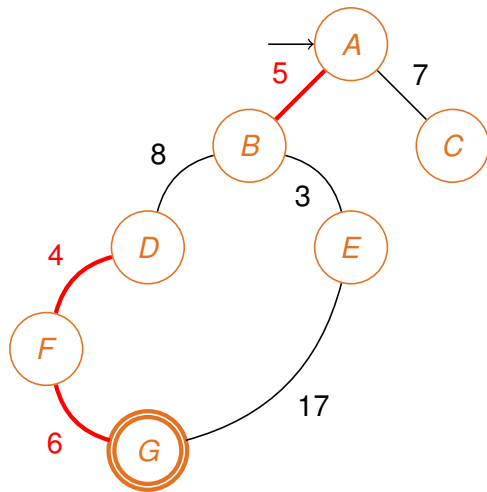
- ▶ G is now the **cheapest** in the frontier ($g=23$): we expand it, and **it is the goal!**
- ▶ Notice how we found a **better** path to G *before* ever confirming it as the solution.



Uniform-Cost Search (Dijkstra)

Uninformed search strategies

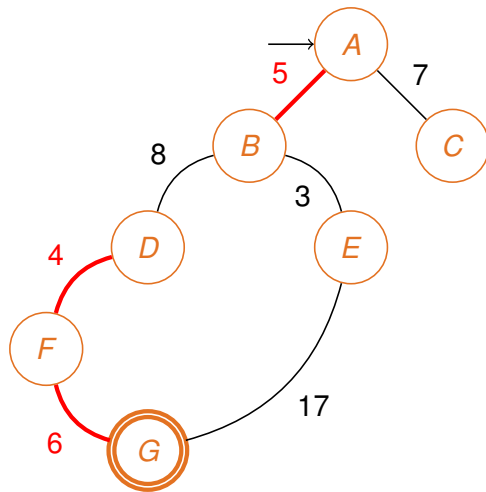
- **Cost optimal:** always finds the **cheapest** solution ($g=23$)...



Uniform-Cost Search (Dijkstra)

Uninformed search strategies

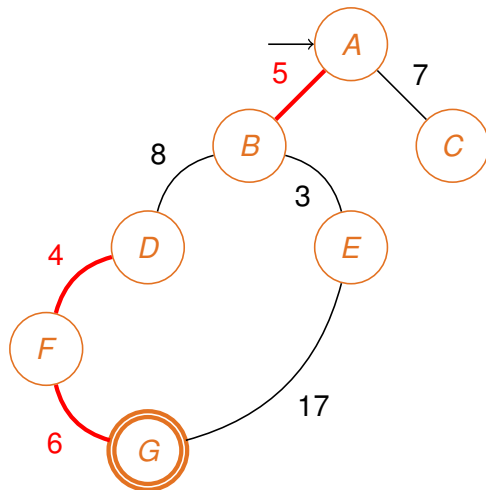
- **Cost optimal:** always finds the **cheapest** solution ($g=23$)...
- ... unlike BFS, which found $A \rightarrow B \rightarrow E \rightarrow G$ at cost 25!



Uniform-Cost Search (Dijkstra)

Uninformed search strategies

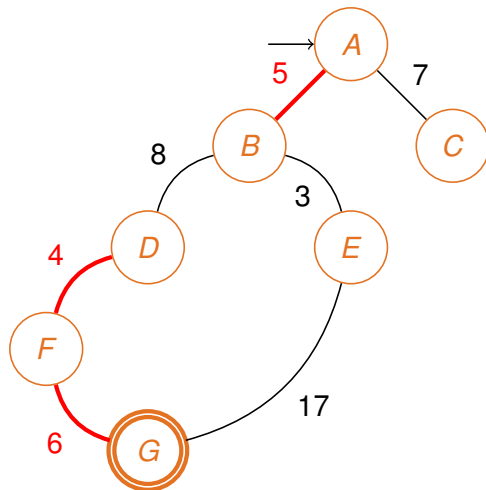
- **Cost optimal:** always finds the **cheapest** solution ($g=23$)...
- ... unlike BFS, which found $A \rightarrow B \rightarrow E \rightarrow G$ at cost 25!
- **Complete**, as long as all action costs are $> \epsilon > 0$



Uniform-Cost Search (Dijkstra)

Uninformed search strategies

- ▶ **Cost optimal:** always finds the **cheapest** solution ($g=23$)...
- ▶ ... unlike BFS, which found $A \rightarrow B \rightarrow E \rightarrow G$ at cost 25!
- ▶ **Complete**, as long as all action costs are $> \epsilon > 0$
- ▶ Time and space complexity: $\mathcal{O}(b^{1+\lceil C^*/\epsilon \rceil})$, where C^* is the cost of the optimal solution



Uninformed search strategies

- ▶ They systematically navigate the search space blindly—not questioning where the goal may be in the space.
- ▶ The search space is often very large.

Uninformed search strategies

- ▶ They systematically navigate the search space blindly—not questioning where the goal may be in the space.
- ▶ The search space is often very large.

Why not being *smarter* about it?

Section 4

Informed search strategies

Heuristic search

Informed search strategies

To take *better informed decisions*, we can use a domain-specific hint about how “desirable” a state can be.

Heuristic search

Informed search strategies

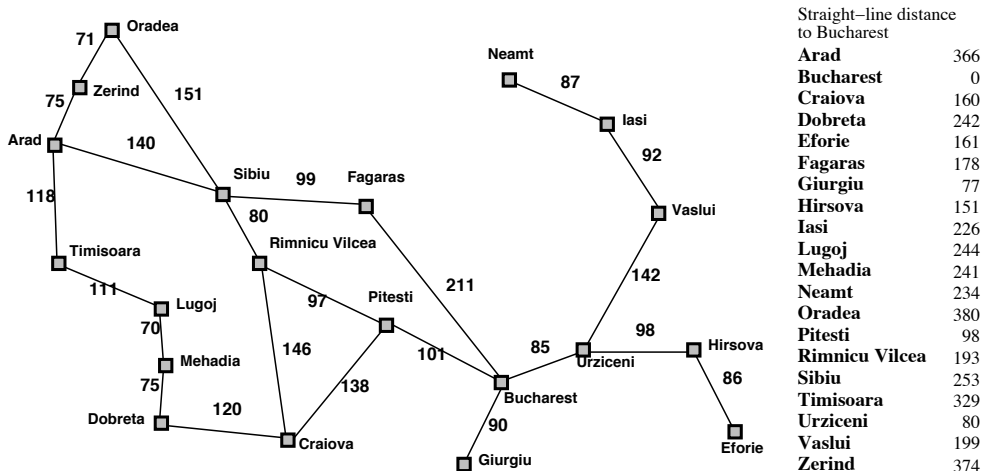
To take *better informed decisions*, we can use a domain-specific hint about how “desirable” a state can be.

This is usually done by using a **heuristic function** $h(n)$, where $h : S \rightarrow \mathbb{R}$, i.e., a *guessing function* about an estimated remaining cost to the goal.

Heuristic example: Romania

Informed search strategies

Using h as the straight line distance to goal:



(Greedy) Best First Search

Informed search strategies

Best first

Choose always the best of your expectations (cheapest estimate).

(Greedy) Best First Search

Informed search strategies

Best first

Choose always the best of your expectations (cheapest estimate).

Yet again, same idea:

(Greedy) Best First Search

Informed search strategies

Best first

Choose always the best of your expectations (cheapest estimate).

Yet again, same idea:

1. Start

(Greedy) Best First Search

Informed search strategies

Best first

Choose always the best of your expectations (cheapest estimate).

Yet again, same idea:

1. Start
2. Check for goal

(Greedy) Best First Search

Informed search strategies

Best first

Choose always the best of your expectations (cheapest estimate).

Yet again, same idea:

1. Start
2. Check for goal
3. Expand and update frontier

(Greedy) Best First Search

Informed search strategies

Best first

Choose always the best of your expectations (cheapest estimate).

Yet again, same idea:

1. Start
2. Check for goal
3. Expand and update frontier
4. Choose **the best** of the estimates

Greedy Best-First Search: the Romania example

Informed search strategies

Using h_{SLD} , the straight-line distance to Bucharest: we start at *Arad* ($h=366$) — the initial state.

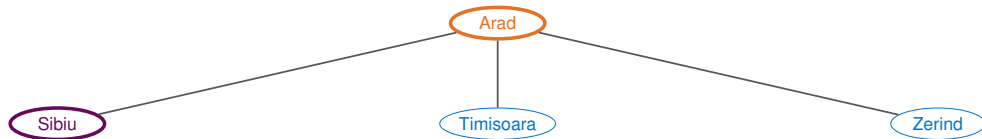


$frontier = \{Arad:366\}$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Greedy Best-First Search: the Romania example

Informed search strategies



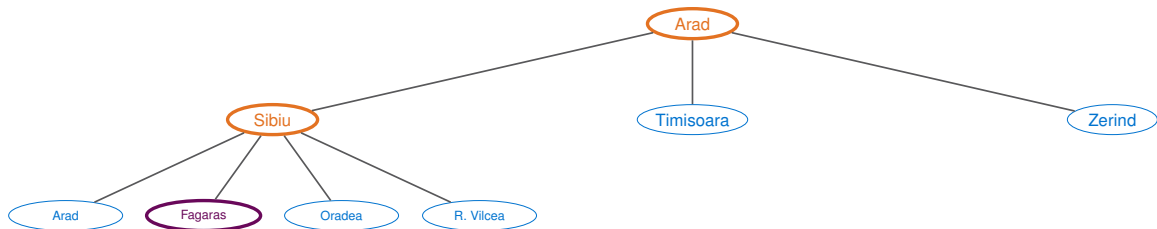
Arad expanded: *Sibiu* ($h=253$) is closest to the goal — expand *Sibiu* next.

frontier = { *Sibiu*:253, *Timisoara*:329, *Zerind*:374 }

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Greedy Best-First Search: the Romania example

Informed search strategies



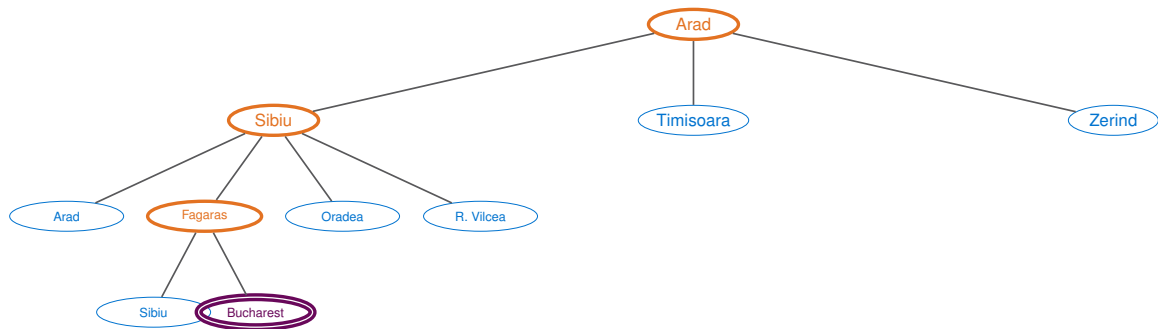
Sibiu expanded: *Fagaras* ($h=176$) is now closest — expand *Fagaras* next. Notice *Arad* reappears, at $h=366$: a **repeated state**!

frontier = { *Fagaras*:176, *R. Vilcea*:193, *Timisoara*:329, *Zerind*:374, *Arad*:366, *Oradea*:380 }

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Greedy Best-First Search: the Romania example

Informed search strategies



Fagaras expanded: *Bucharest* is generated with $h=0$ — it's the **goal**!

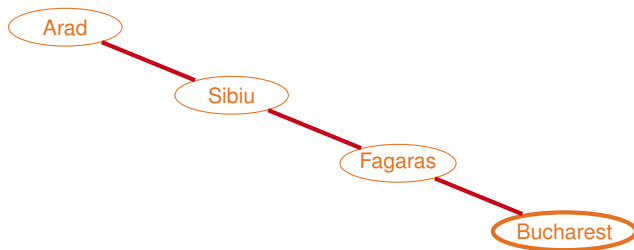
frontier =

{*Bucharest*:0, *R. Vilcea*:193, *Timisoara*:329, *Zerind*:374, *Sibiu*:253, *Arad*:366, *Oradea*:380}

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

Greedy Best-First Search: the Romania example

Informed search strategies



Bucharest is chosen next, and confirmed as the **goal**. Solution:

Arad → *Sibiu* → *Fagaras* → *Bucharest* (450 miles).

This never expands a node off the final path — but it's **not optimal**: the route via Rimnicu Vilcea and Pitesti is 32 miles shorter (418 miles).

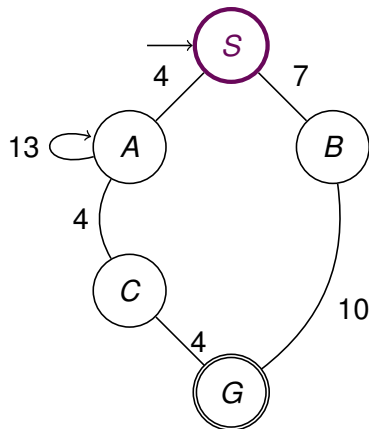
(Greedy) Best First Search

Informed search strategies

With the following estimated distances to the goal:

- ▶ $h(A) = 3$
- ▶ $h(B) = 3$
- ▶ $h(C) = 3$
- ▶ $h(G) = 0$

We choose alphabetically in case of a tie.



● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

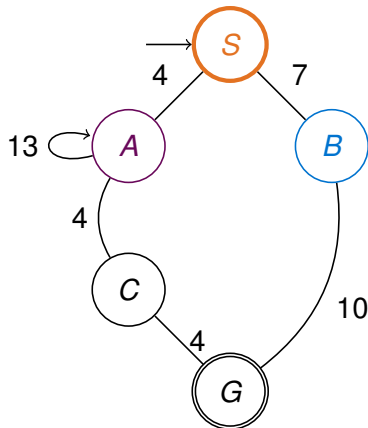
(Greedy) Best First Search

Informed search strategies

With the following estimated distances to the goal:

- ▶ $h(A) = 3$
- ▶ $h(B) = 3$
- ▶ $h(C) = 3$
- ▶ $h(G) = 0$

We choose alphabetically in case of a tie.



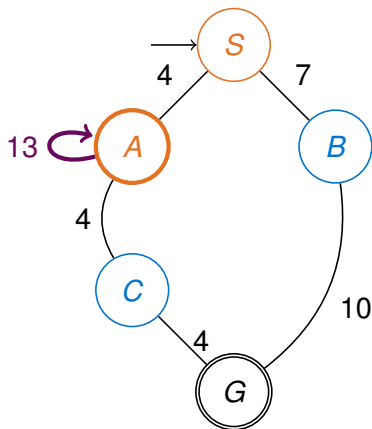
S expanded: A and B tie at $h=3$. Alphabetically, A wins: expand A next.

$frontier = \{A:3, B:3\}$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

(Greedy) Best First Search

Informed search strategies



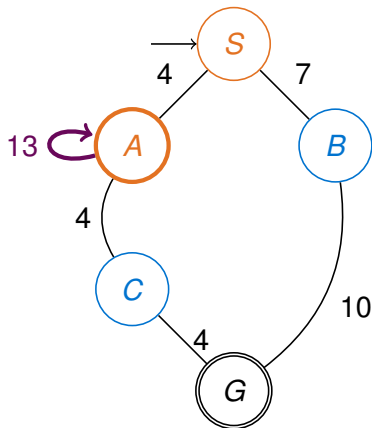
A expanded: this generates C ($h=3$) — but **also regenerates A itself**, via that loop! B, A (again), and C are all tied at $h=3$.

frontier = { $A_{\text{loop}}:3$, $B:3$, $C:3$ } — alphabetically, A wins *again*: expand A next.

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

(Greedy) Best First Search

Informed search strategies



We expand **A again** — and it generates the exact same things: the loop back to A, and C. Nothing changes!

$frontier = \{A_{loop}:3, B:3, C:3\}$ — unchanged. With tree-like search, this repeats **forever**.

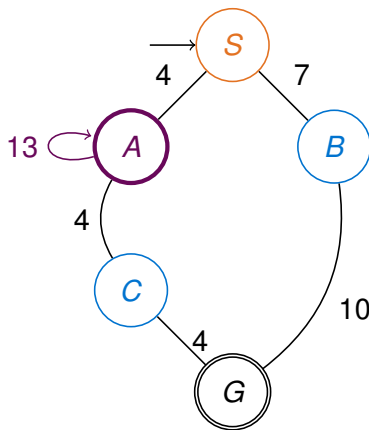
● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

(Greedy) Best First Search

Informed search strategies

With those estimated distances to the goal:

- ▶ We have a **cycle**!
- ▶ Tree-like search would not make it past *A*
- ▶ By adding memory we make it smarter. Still, space complexity increases.
- ▶ **Always Complete** in finite spaces with no loops (not our case)
- ▶ **Might not be cost optimal** (See Romania example!)



A* search

Informed search strategies

What if we consider the cost *and* the heuristic?

A* search

Informed search strategies

What if we consider the cost *and* the heuristic?

Our new **evaluation function** (criterion) will consider both things:

$$f(n) = g(n) + h(n)$$

where

A* search

Informed search strategies

What if we consider the cost *and* the heuristic?

Our new **evaluation function** (criterion) will consider both things:

$$f(n) = g(n) + h(n)$$

where

- ▶ $g(n)$ is the cost we have paid so far to reach n

A* search

Informed search strategies

What if we consider the cost *and* the heuristic?

Our new **evaluation function** (criterion) will consider both things:

$$f(n) = g(n) + h(n)$$

where

- ▶ $g(n)$ is the cost we have paid so far to reach n
- ▶ $h(n)$ is the estimated cost of the node (to the goal)

A* search

Informed search strategies

What if we consider the cost *and* the heuristic?

Our new **evaluation function** (criterion) will consider both things:

$$f(n) = g(n) + h(n)$$

where

- ▶ $g(n)$ is the cost we have paid so far to reach n
- ▶ $h(n)$ is the estimated cost of the node (to the goal)
- ▶ $f(n)$ is then the estimated cost of the cheapest solution through n to the goal

A^* search: the Romania example

Informed search strategies

We start at *Arad*: $g=0$, $h=366$, so $f=366$ — the initial state.



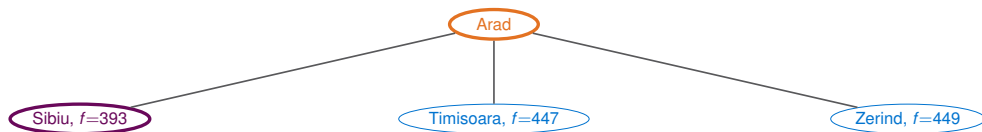
Arad

$frontier = \{Arad:366\}$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search: the Romania example

Informed search strategies



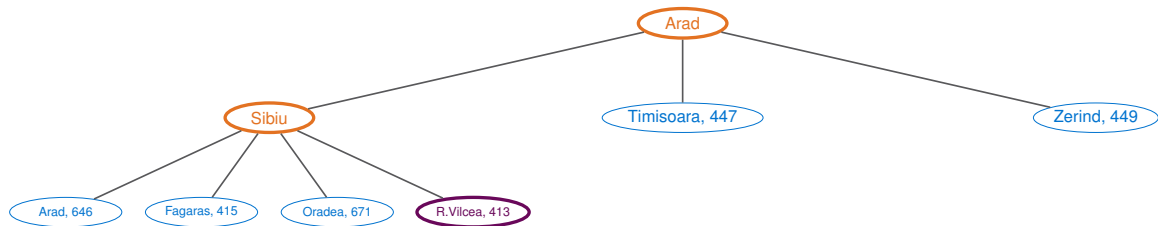
Arad expanded: *Sibiu* has the lowest $f=140+253=393$ — expand *Sibiu* next.

frontier = { *Sibiu*:393, *Timisoara*:447, *Zerind*:449 }

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search: the Romania example

Informed search strategies



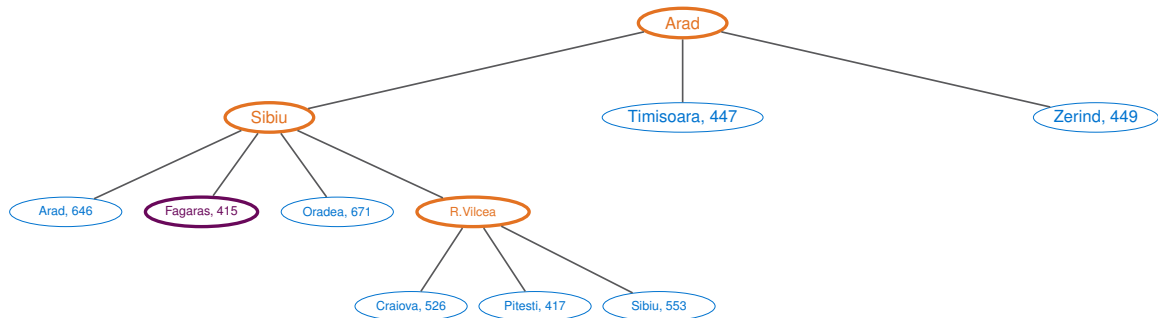
Sibiu expanded: *Rimnicu Vilcea* has the lowest $f=220+193=413$ — expand it next.

frontier = { *R. Vilcea*:413, *Fagaras*:415, *Timisoara*:447, *Zerind*:449, *Arad*:646, *Oradea*:671 }

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search: the Romania example

Informed search strategies



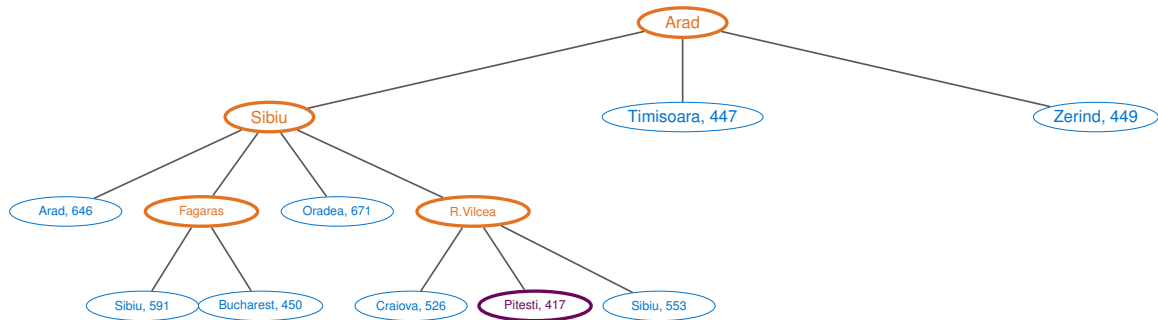
Rimnicu Vilcea expanded: *Fagaras* ($f=415$) is still the lowest overall — expand *Fagaras* next.

$frontier = \{Fagaras:415, Pitesti:417, Timisoara:447, Zerind:449, Craiova:526, Sibiu:553, Arad:646, Oradea:671\}$

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search: the Romania example

Informed search strategies



Fagaras expanded: *Bucharest* is generated ($f=450$), but it's **not** the cheapest yet! *Pitesti* ($f=417$) is: expand it next.

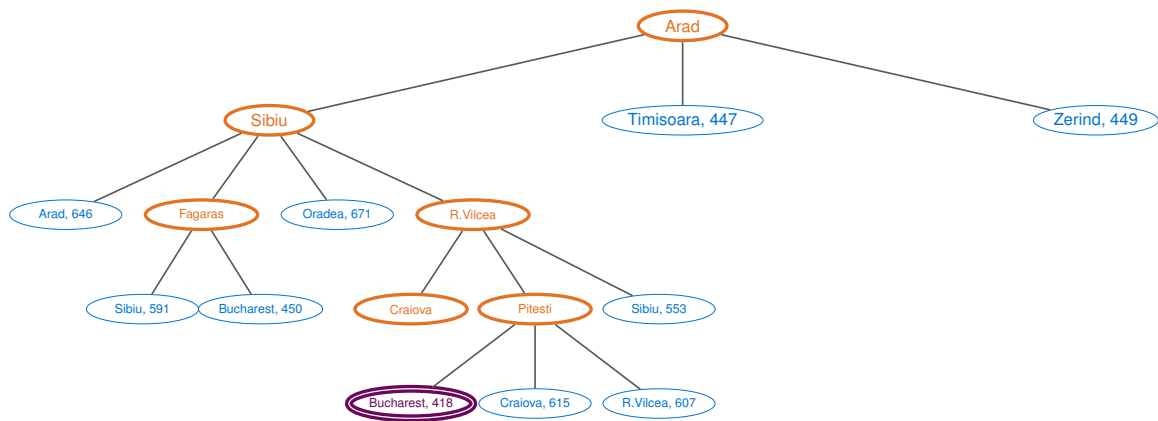
frontier =

{*Pitesti*:417, *Timisoara*:447, *Zerind*:449, *Bucharest*:450, *Craiova*:526, *Sibiu*:553, *Sibiu*:591, *Arad*:646, *Oradea*:671}

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search: the Romania example

Informed search strategies

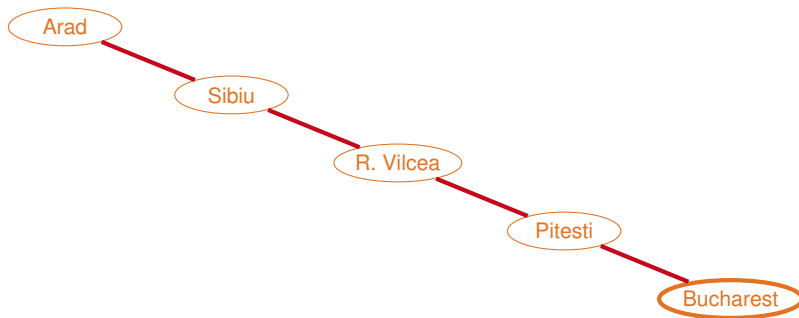


Pitesti expanded: a **cheaper** path to *Bucharest* is found, $f=418 < 450$! It **replaces** the old entry: *Bucharest* is now cheapest — select and confirm as **goal**.

● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search: the Romania example

Informed search strategies



Bucharest is confirmed as the **goal**. Solution:

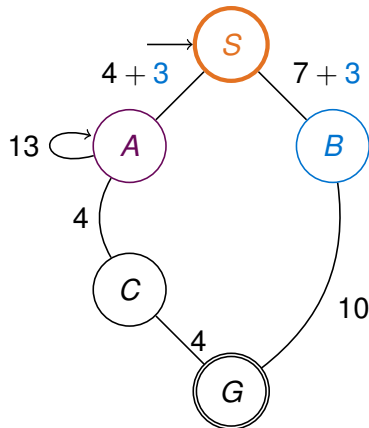
Arad → *Sibiu* → *Rimnicu Vilcea* → *Pitesti* → *Bucharest* (418 miles) — the truly **optimal** route.

A* search

Informed search strategies

With the following estimated distances to the goal:

- ▶ $h(A) = 3$
- ▶ $h(B) = 3$
- ▶ $h(C) = 3$
- ▶ $h(G) = 0$



We expand S : A gets $f=4+3=7$, B gets $f=7+3=10$. A is cheaper: expand A next.
 $frontier = \{A:7, B:10\}$

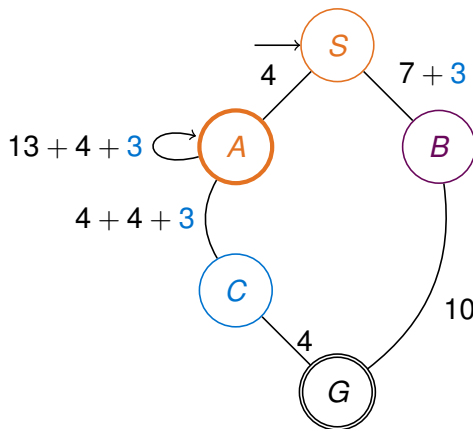
● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search

Informed search strategies

With the following estimated distances to the goal:

- ▶ $h(A) = 3$
- ▶ $h(B) = 3$
- ▶ $h(C) = 3$
- ▶ $h(G) = 0$



We expand A : the loop back to A costs $f=20$, C costs $f=11$. But B , still at $f=10$, is cheapest: expand B next.

$frontier = \{B:10, C:11, A_{loop}:20\}$

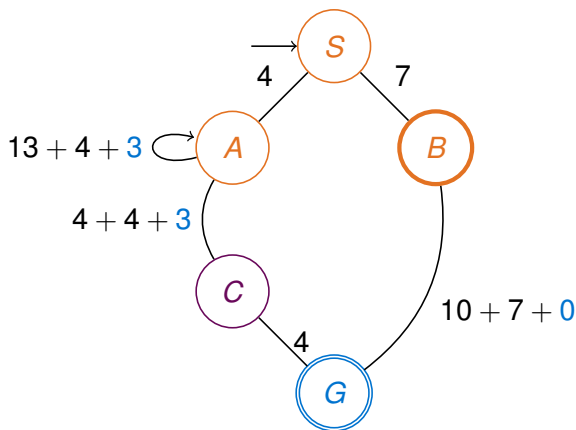
● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search

Informed search strategies

With the following estimated distances to the goal:

- ▶ $h(A) = 3$
- ▶ $h(B) = 3$
- ▶ $h(C) = 3$
- ▶ $h(G) = 0$



We expand B : this generates G , a candidate goal, at $f=17$. But C , at $f=11$, is still cheaper — we do **not** stop: expand C next.

$frontier = \{C:11, G:17, A_{loop}:20\}$

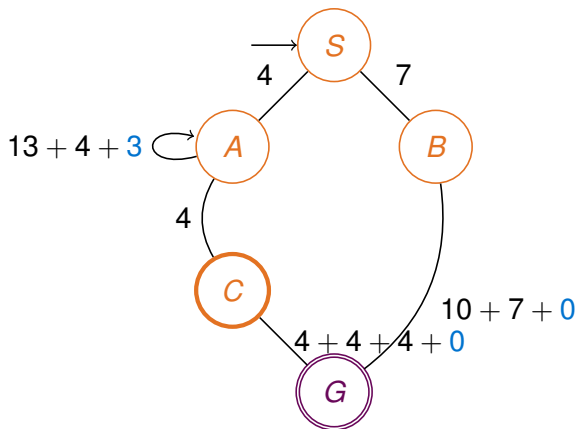
● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search

Informed search strategies

With the following estimated distances to the goal:

- ▶ $h(A) = 3$
- ▶ $h(B) = 3$
- ▶ $h(C) = 3$
- ▶ $h(G) = 0$



We expand C : a **cheaper path** to G is found, $f=12 < 17$! It **replaces** the old entry. G is now cheapest.

$frontier = \{G:12, A_{loop}:20\}$

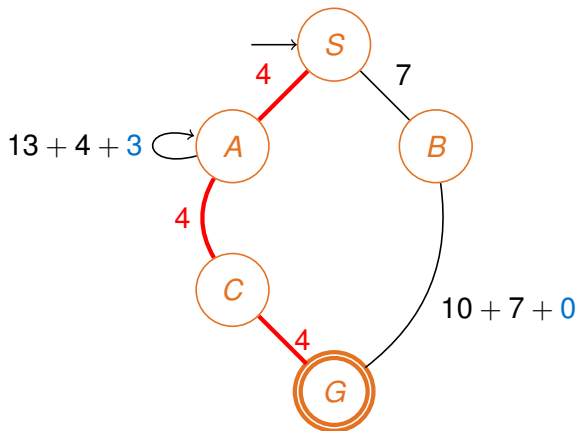
● next to expand ● expanding now (bold border) ● in frontier ○ not yet reached

A* search

Informed search strategies

With the those estimated distances to the goal:

- We have found the goal!

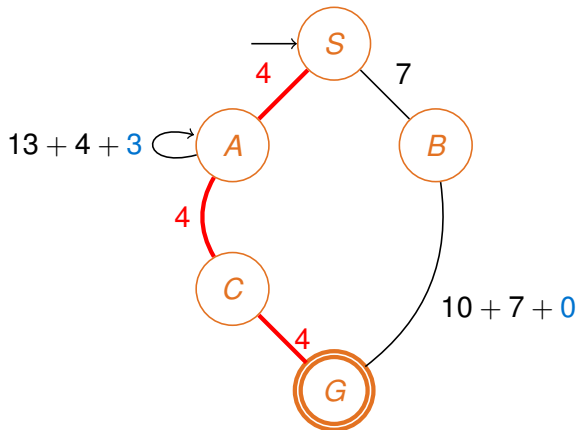


A* search

Informed search strategies

With the those estimated distances to the goal:

- ▶ We have found the goal!
- ▶ It is **complete** for positive costs, within a finite state space and an existing solution.

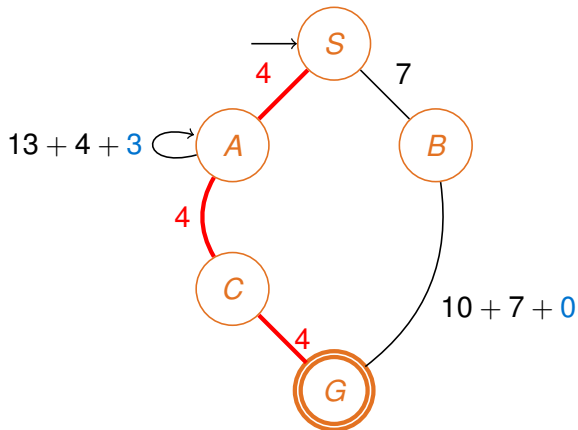


A* search

Informed search strategies

With the those estimated distances to the goal:

- ▶ We have found the goal!
- ▶ It is **complete** for positive costs, within a finite state space and an existing solution.
- ▶ it is **cost optimal** if **certain conditions are met**



Friendly reminder

Things to look out for

Implementation details vary a lot, and can be tricky!

- ▶ Is the algorithm checking for redundant paths (**graph search**) or not (**tree-like search**)?
- ▶ Is the goal check performed **early** (when a node is *generated*) or **late** (when a node is *expanded*)?
- ▶ Is the algorithm **storing** all **reached states**, or **reconstructing the path** from a chain of parent nodes?

Read the book!

To become familiar with the algorithms and their implementations details, you should read the book. These slides are not a replacement for the book; they are a summary of the most important points.

Cheatsheet

Things to look out for

Most of the search strategies we cover in this course use the same algorithm to search.⁵ It is just Best-First-Search with different functions to decide which element will be popped out of the priority queue:

Depth-First Search

$$f(n) = -\text{depth}(n)$$

Uniform-Cost Search (Dijkstra)

$$f(n) = g(n)$$

Greedy Best-First Search

$$f(n) = h(n)$$

A* Search

$$f(n) = g(n) + h(n)$$

⁵Except for BFS that has a separate algorithm.